

2022 하반기 기말프로젝트

# 고객 구매데이터로 결혼여부 예측

K2022531 인공지능전공

최혜정





# CONTENTS

## 01. 데이터 탐색

- 고객정보 탐색
- 구매정보 확인

## 02. 데이터분석

- 타겟변수 확인
- 파생변수 생성
- 변수 표준화 및 정규화
- 변수선택

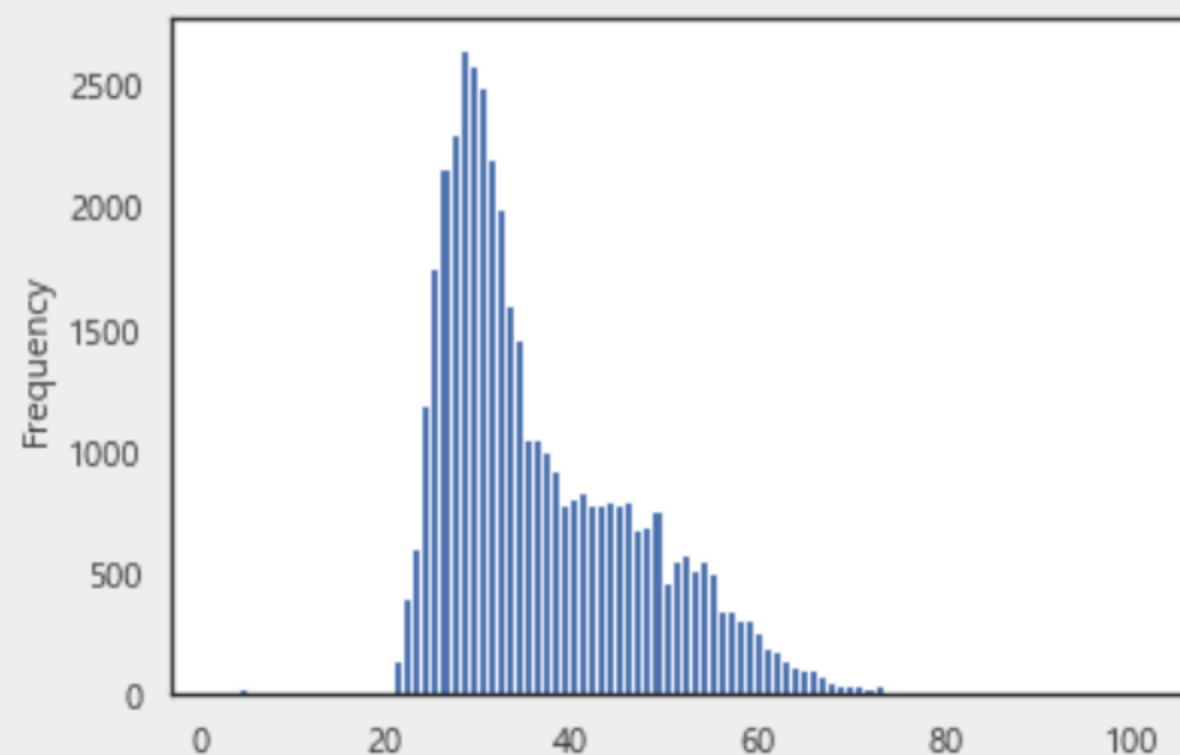
## 03. 모델링 및 평가

- 분류모델링
- 모델 평가
- 최종 모델 선정

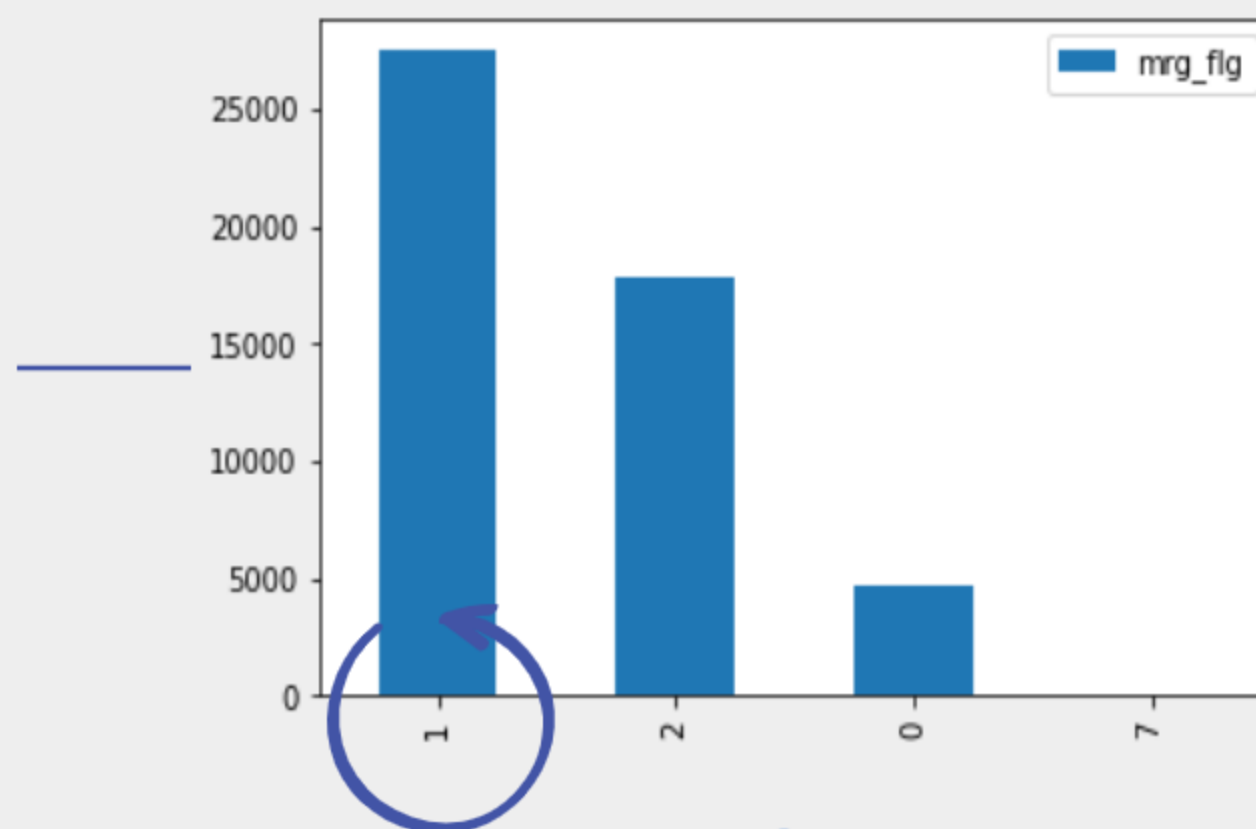
## 04. 느낀점

## 고객정보 데이터

Customer 테이블은 고객 아이디, 고객성별, 생일, 결혼여부, 주거형태, 주거현황, 취미, 직업, 고객유형, 주구매 시간대의 컬럼으로 고객정보를 나타냄.



고객 나이대

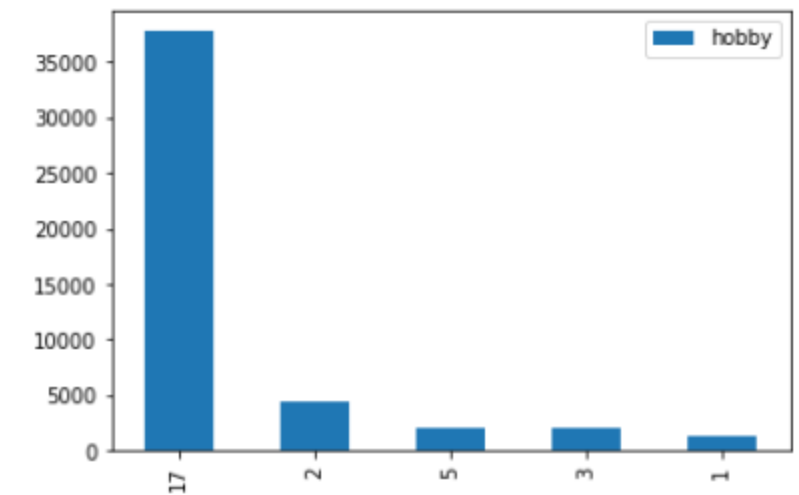
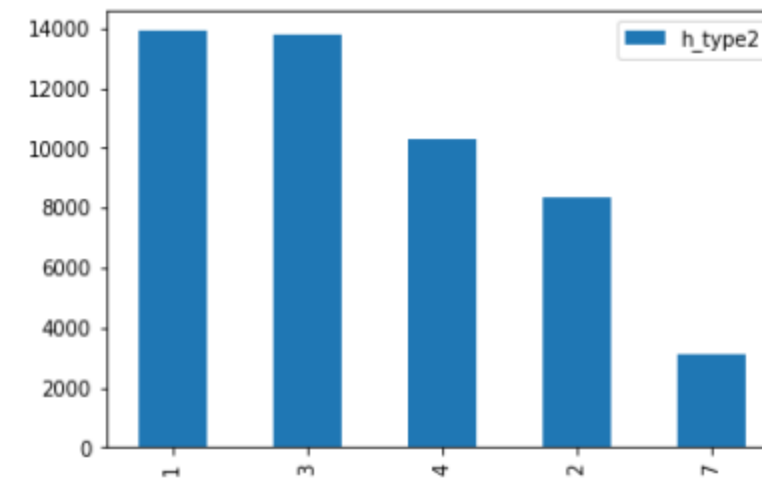
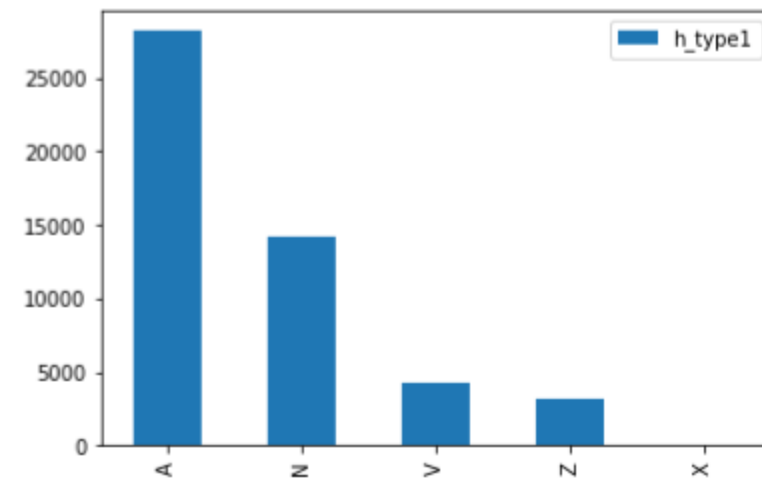
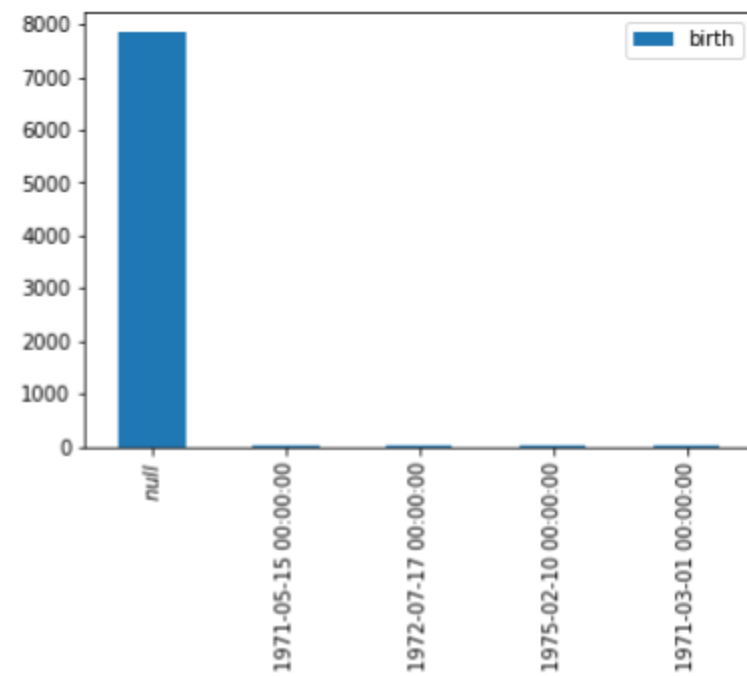


결혼 여부



# 고객정보 데이터

shape (49994, 11)



## 생일, 주거형태, 주거현황, 취미 데이터

생일 컬럼엔 null 값이 8000개 정도 들어가있는 것을 알 수 있음.  
주거형태는 아파트 > 단독주택 > 빌라 > 기타(분류안됨) > 기타

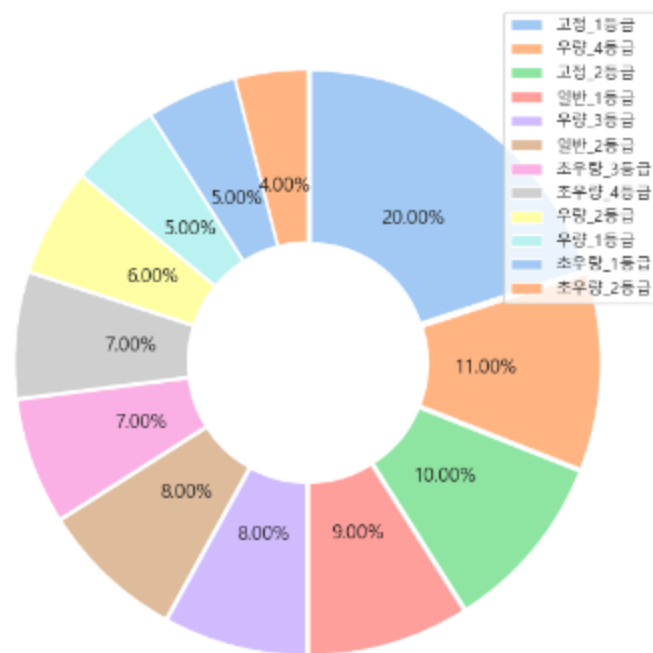
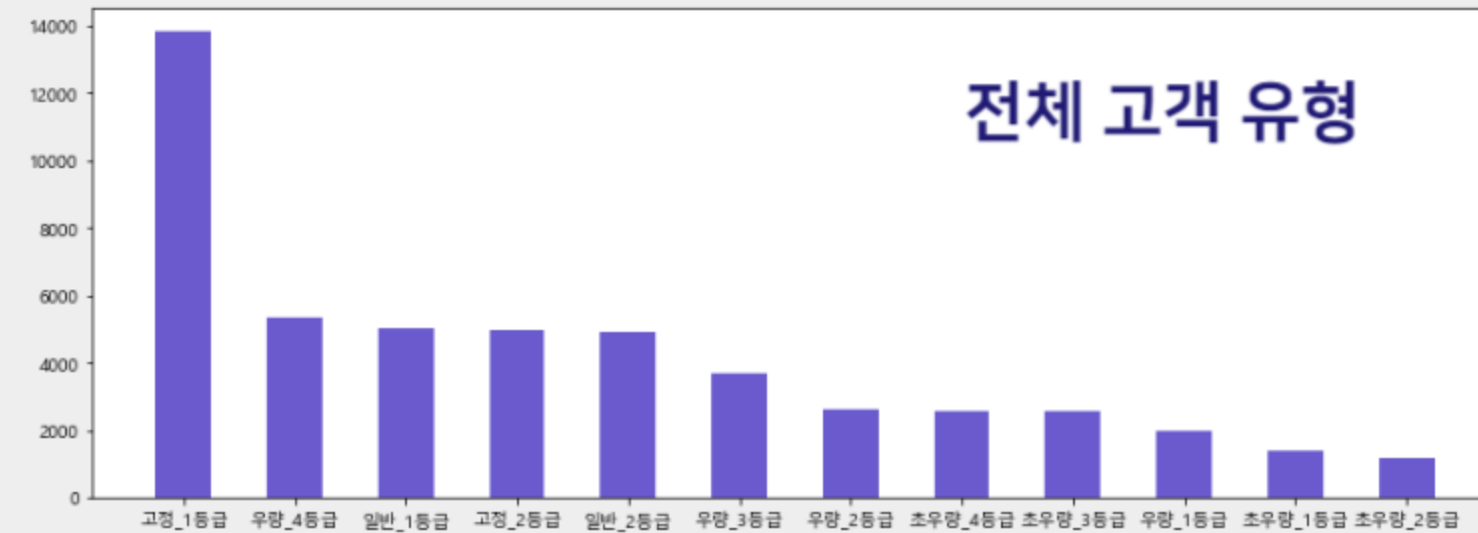
주거현황은 본인소유 - 부모소유 > 전세 > 배우자소유 > 기타 순서.  
취미는 기타 > 스포츠 > 관람 > 여행 > 등산 순서.

## 고객정보 데이터

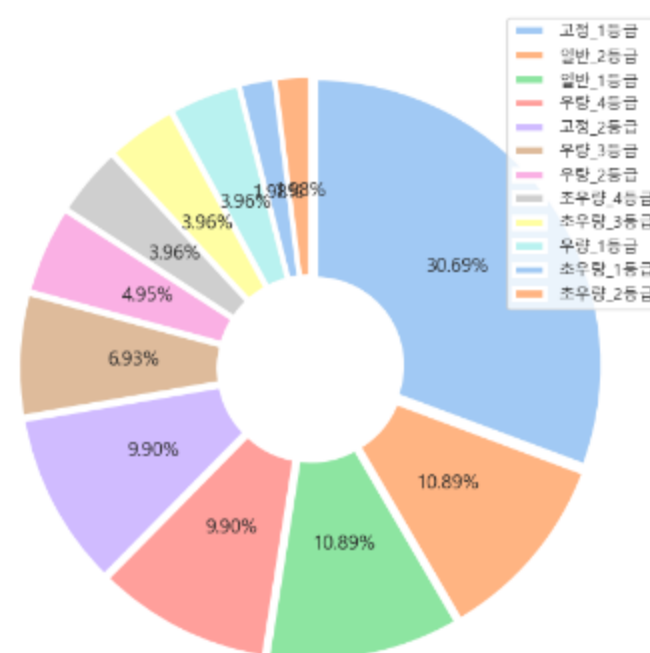
고객소유형은 초우량1등급에서부터 일반2등급까지 12등급으로 나뉘져 있음.

초우량 -> 우량 -> 고정 -> 일반으로 분리됨.

남성, 여성 관계없이 고정 1등급의 고객이 가장 많은 것을 알 수 있음.



남성 고객유형



여성 고객유형

> 남성은 고정 1등급 다음으로 우량 4등급의 고객이 많음.

> 여성은 고정1등급 다음으로 일반 2등급, 일반 1등급 의 고객이 많음.



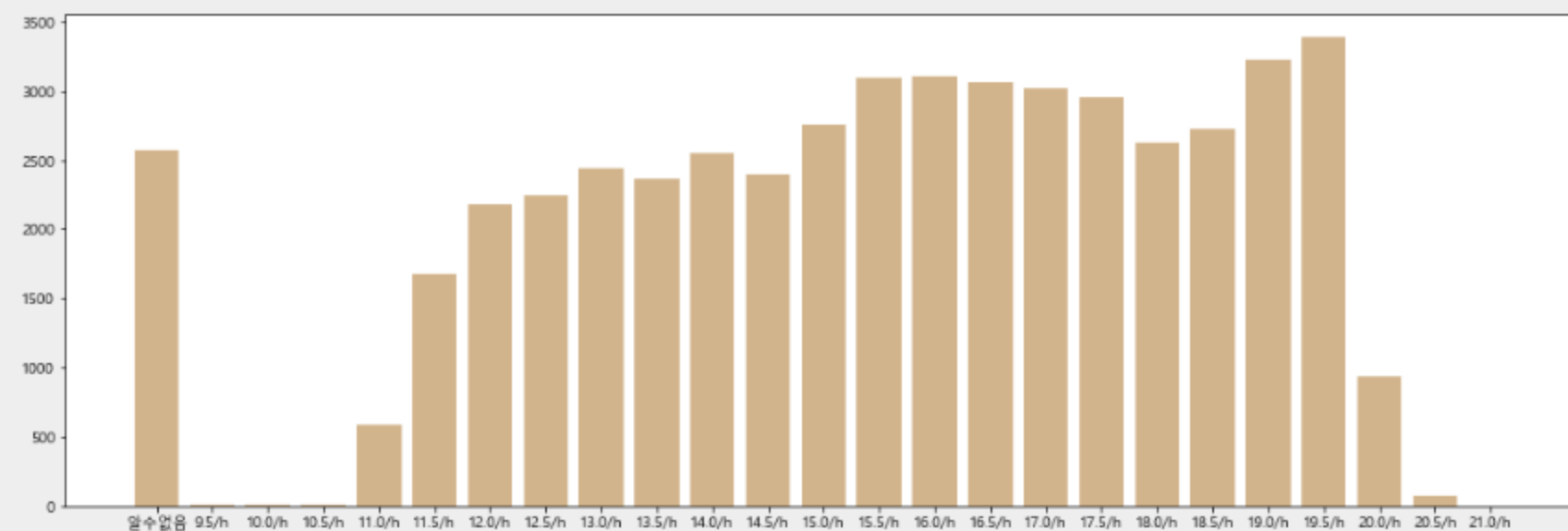
## 고객정보 데이터

구매시간대 오후 - 유독 집중되는 시간대는 없음.

가장 높은 구매시간대 39로, 오후 7시 30분

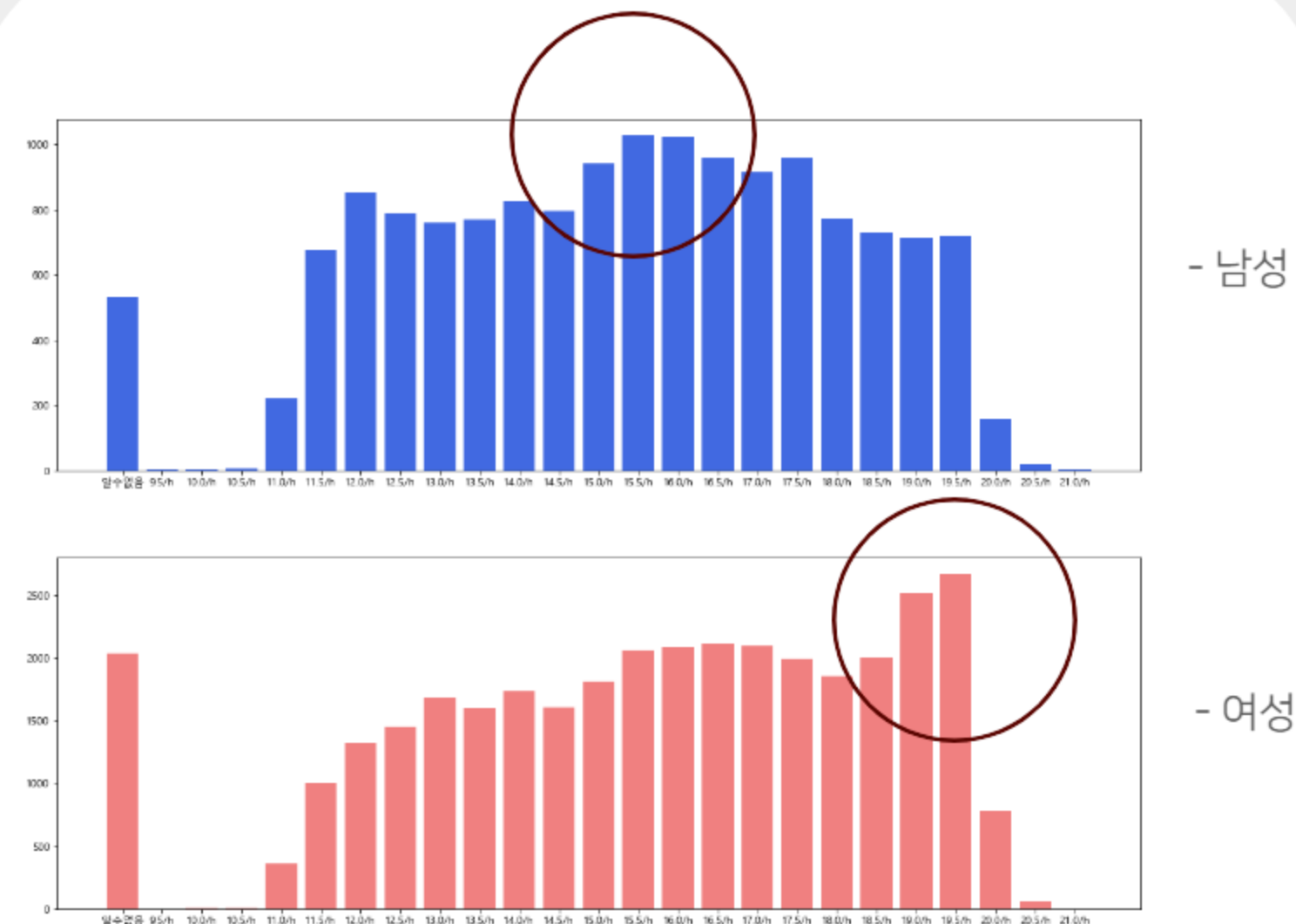
9시반부터 조금씩 있지만, 주구매 시간대는 11시 부터 시작됨을 알 수 있음.

후 8시 이후로 주구매 고객이 줄어들면서 8시 30분 이후로는 거의 없음.



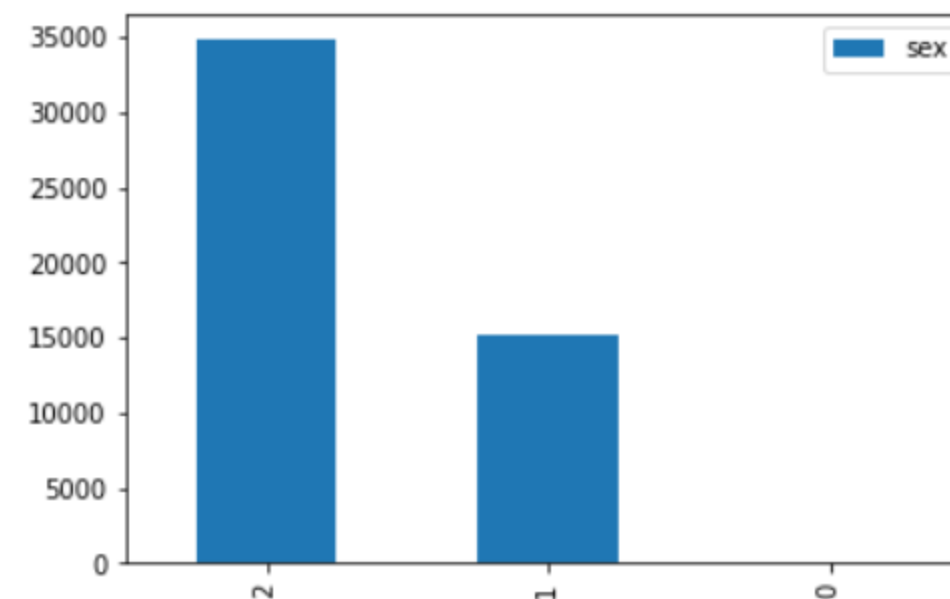
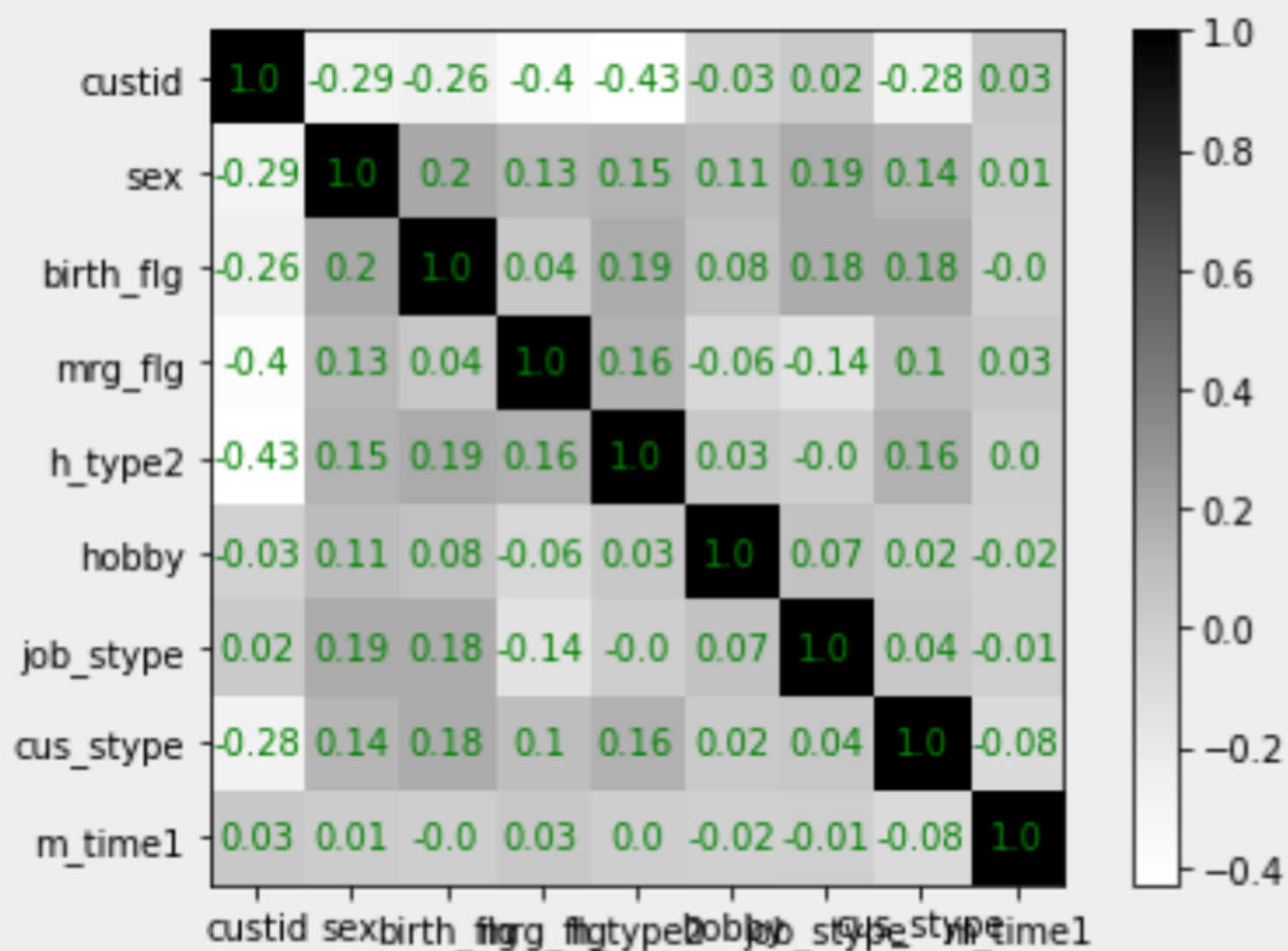
### 전체 고객 구매시간대

( 19시 30분 > 19시 > 16시 > 15시 30분 > 16시 30분 )

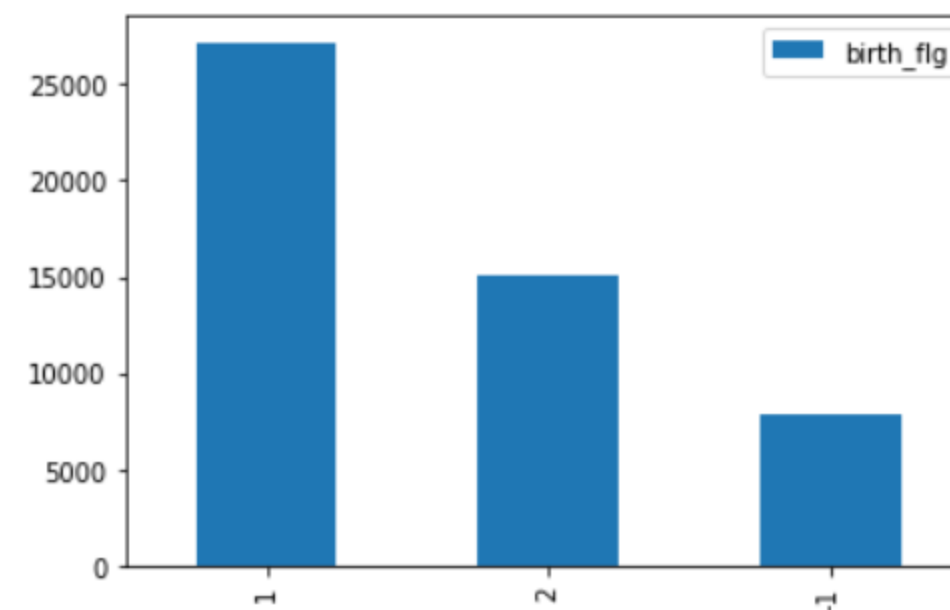


# 고객정보 데이터

가장 상관계수가 높은 값은 0.2으로 상관관계가 낮으며, 변수쌍은 sex - birth\_flg



성별 분포



생일 구분 분포

## 구매정보 데이터

Transaction 데이터는 구매정보 데이터로, 판매 일자, 고객아이디, 상품코드, 브랜드코드, 팀코드, 파트코드, 상품군코드, 코너코드, 구매액, 할인액, 실구매액으로 이루어진 테이블.

|      |       | custid | per   |
|------|-------|--------|-------|
| year | month |        |       |
| 2001 | 4     | 176884 | 10.25 |
| 2000 | 7     | 167403 | 9.70  |
|      | 6     | 163624 | 9.48  |
|      | 10    | 160938 | 9.32  |
|      | 12    | 148955 | 8.63  |
|      | 5     | 144474 | 8.37  |
| 2001 | 3     | 136763 | 7.92  |
|      | 1     | 135967 | 7.88  |
| 2000 | 9     | 133188 | 7.71  |
|      | 11    | 124906 | 7.23  |
|      | 8     | 118948 | 6.89  |
| 2001 | 2     | 114380 | 6.63  |

가장 구매가 많이 이루어진 달,  
2001년 4월

|      |       | tot_amt  |
|------|-------|----------|
| year | month |          |
| 2000 | 5     | 35650000 |
|      | 6     | 33620000 |
|      | 7     | 30820000 |
|      | 8     | 19587000 |
|      | 9     | 72000000 |
|      | 10    | 29000000 |
|      | 11    | 12460000 |
|      | 12    | 21240000 |
| 2001 | 1     | 17740000 |
|      | 2     | 13176000 |
|      | 3     | 26300000 |
|      | 4     | 27327500 |

구매액 최대 높은 달,  
2000년 5월



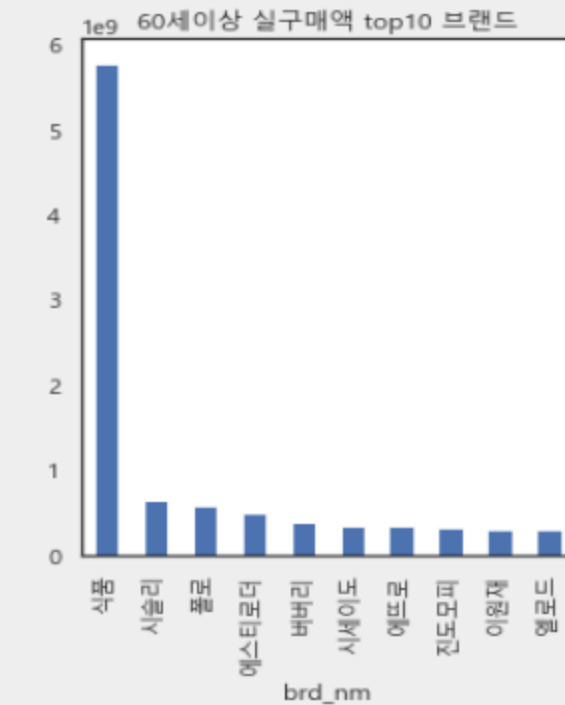
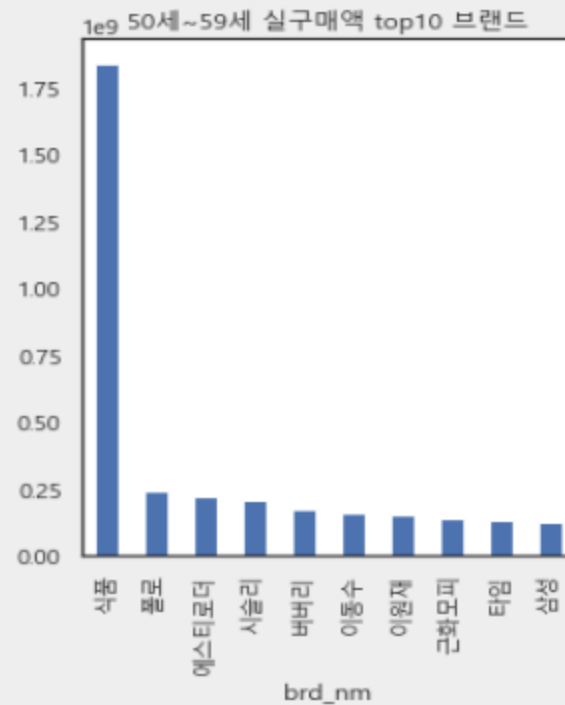
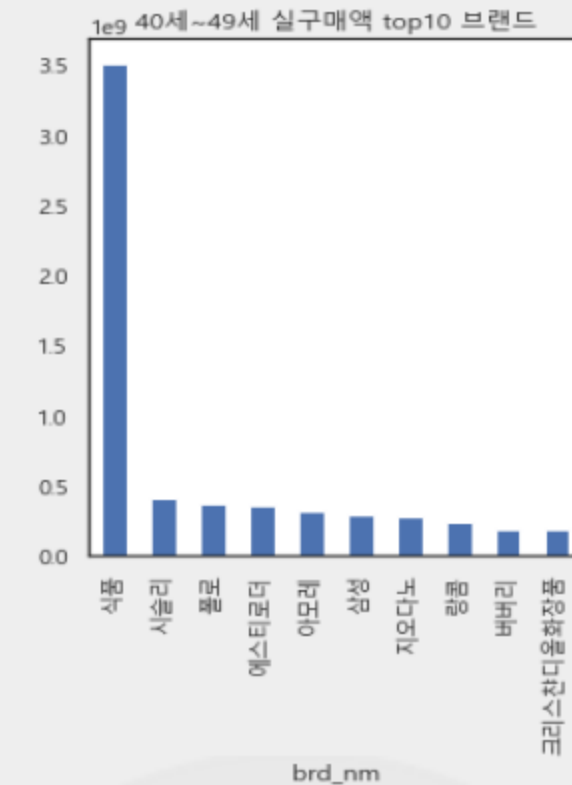
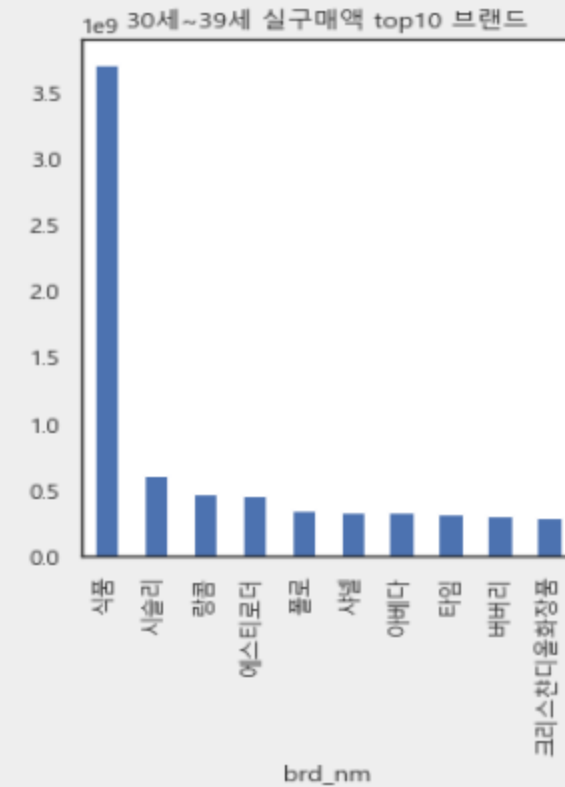
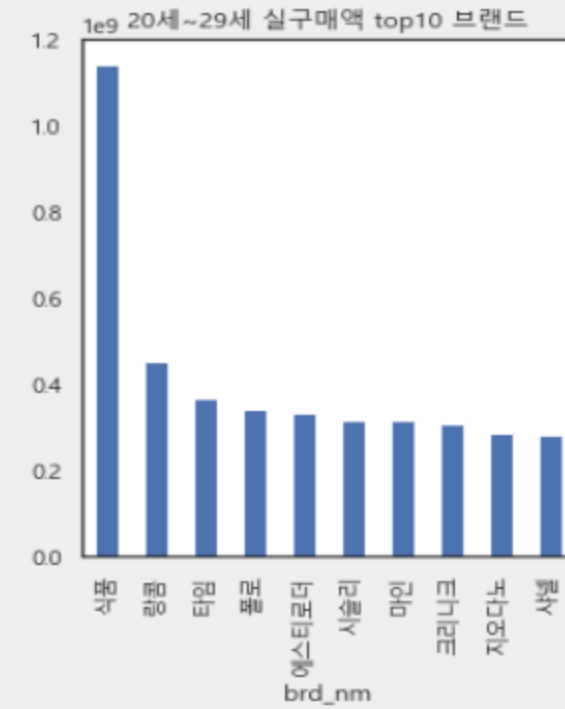
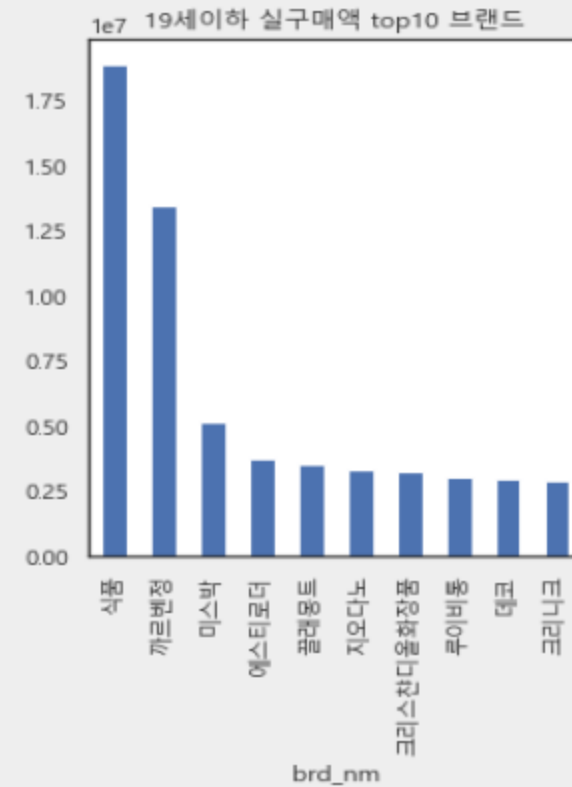
# 구매정보 데이터

나이대별 구매 브랜드 top10

|       | tot_amt   | dis_amt | net_amt   |
|-------|-----------|---------|-----------|
| age_m |           |         |           |
| 1     | 88953.96  | 2994.46 | 85959.50  |
| 2     | 86490.68  | 3439.32 | 83051.36  |
| 3     | 89682.61  | 3076.21 | 86606.41  |
| 4     | 98704.50  | 2974.78 | 95729.72  |
| 5     | 109571.21 | 3360.81 | 106210.40 |

## 나이대별 평균 구매액

1~6 순서대로 [ '19세이하', '20세~29세', '30세~39세', '40세~49세', '50세~59세', '60세이상' ]



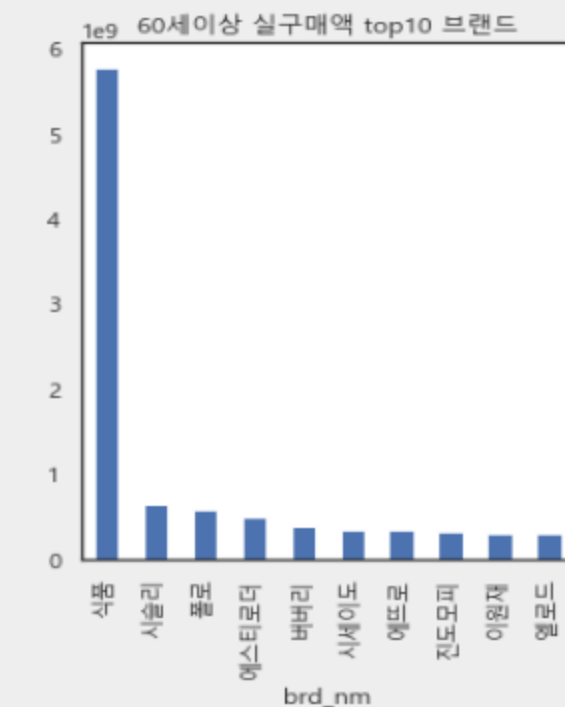
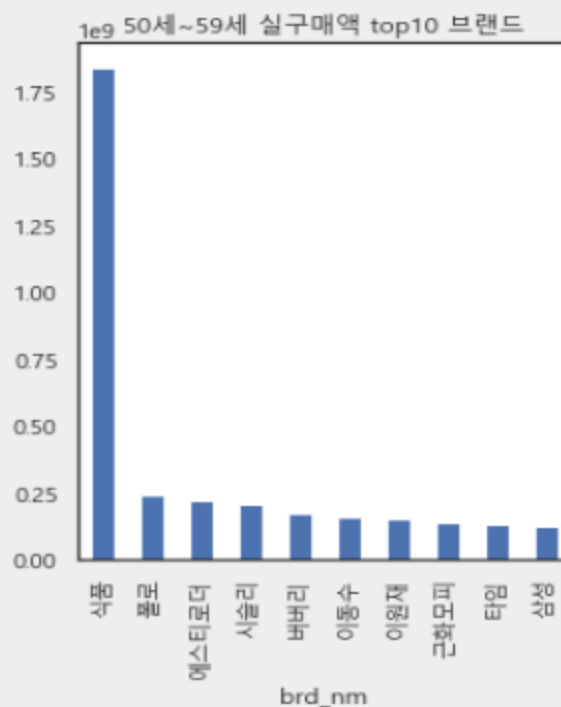
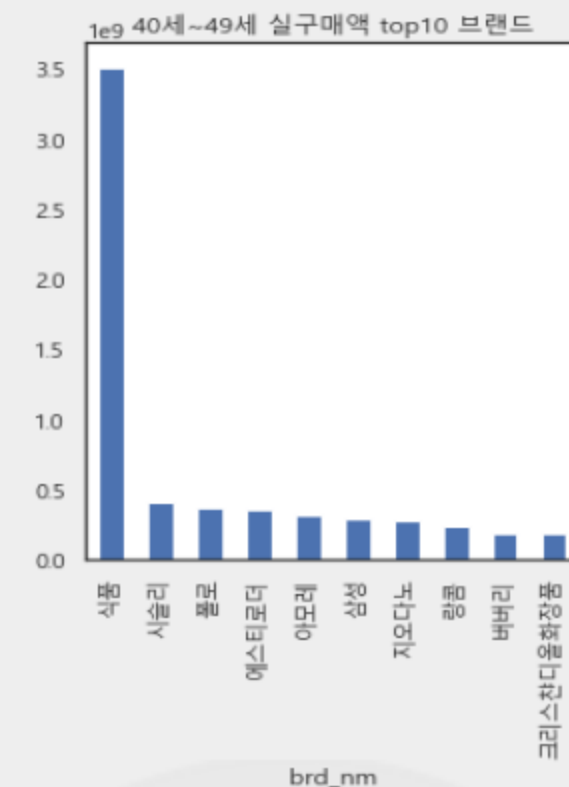
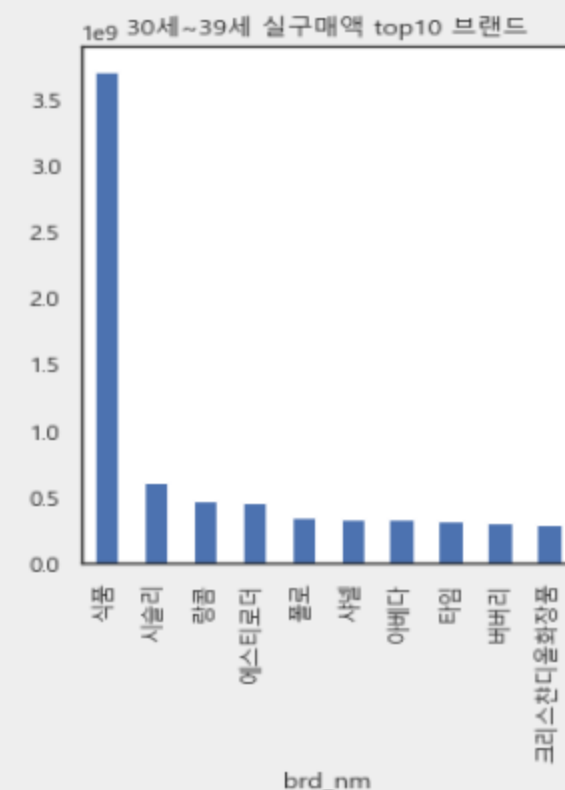
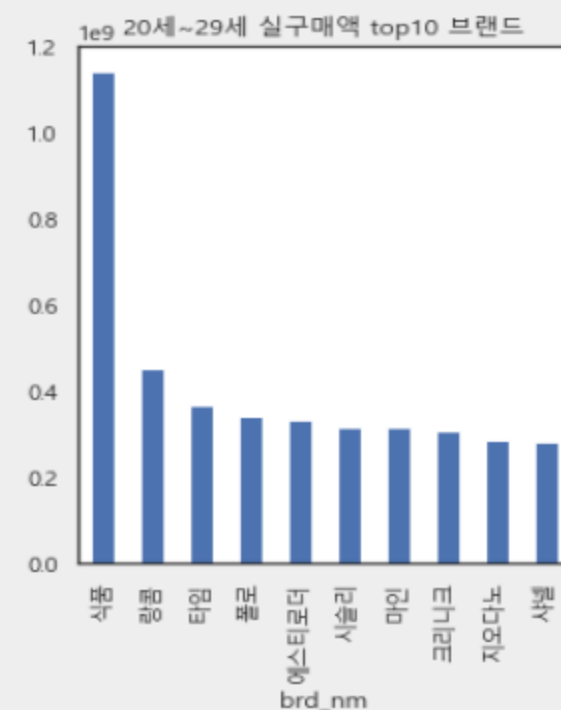
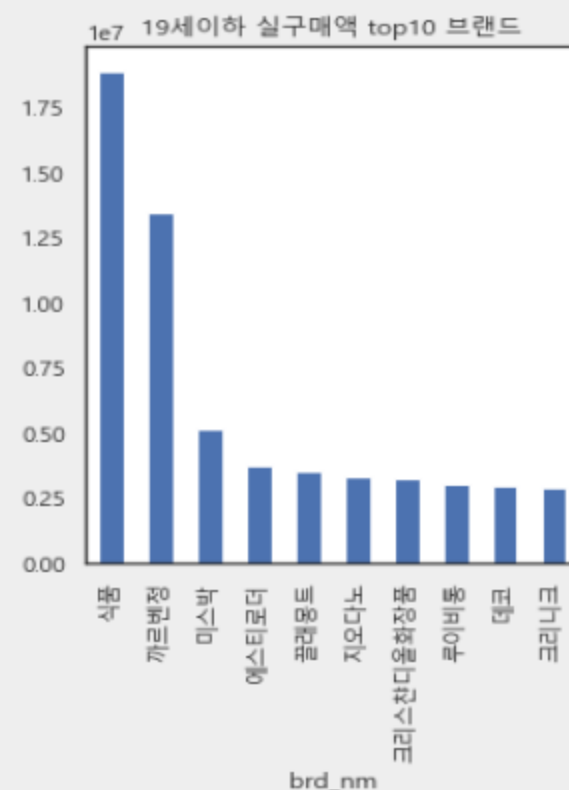
# 구매정보 데이터

나이대별 구매 브랜드 top10

|       | tot_amt   | dis_amt | net_amt   |
|-------|-----------|---------|-----------|
| age_m |           |         |           |
| 1     | 88953.96  | 2994.46 | 85959.50  |
| 2     | 86490.68  | 3439.32 | 83051.36  |
| 3     | 89682.61  | 3076.21 | 86606.41  |
| 4     | 98704.50  | 2974.78 | 95729.72  |
| 5     | 109571.21 | 3360.81 | 106210.40 |

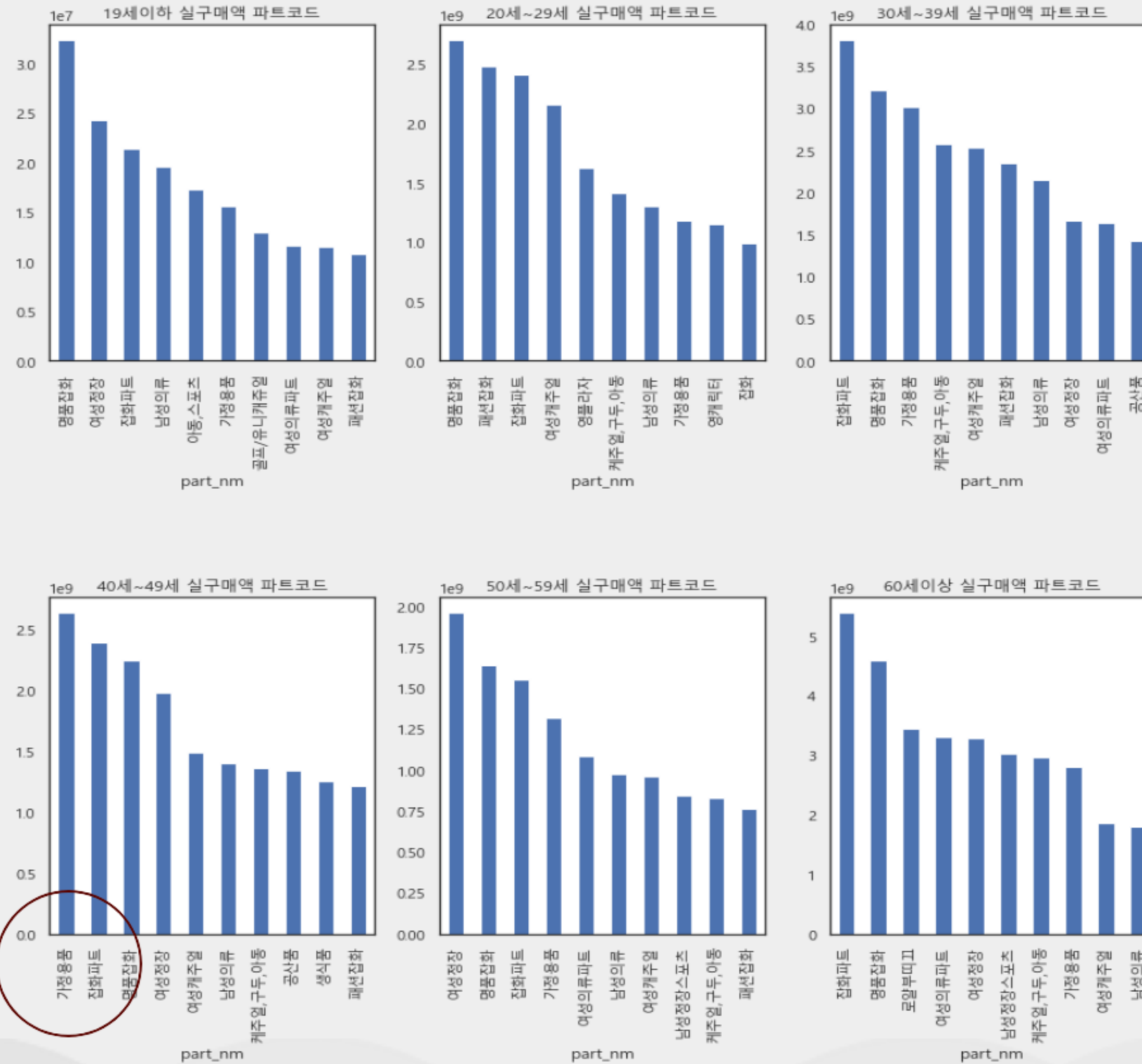
## 나이대별 평균 구매액

1~6 순서대로 [ '19세이하', '20세~29세', '30세~39세', '40세~49세', '50세~59세', '60세이상' ]



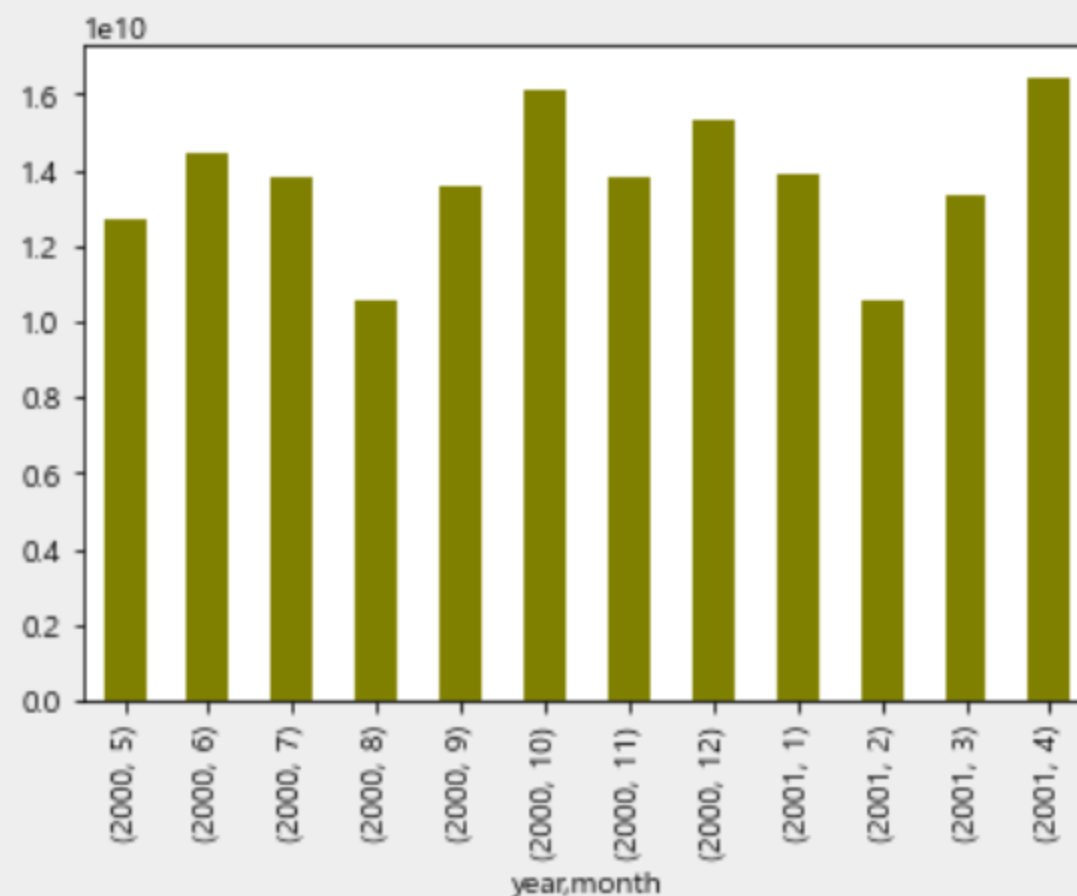
# 구매정보 데이터

나이대별 실구매액 높은 파트코드



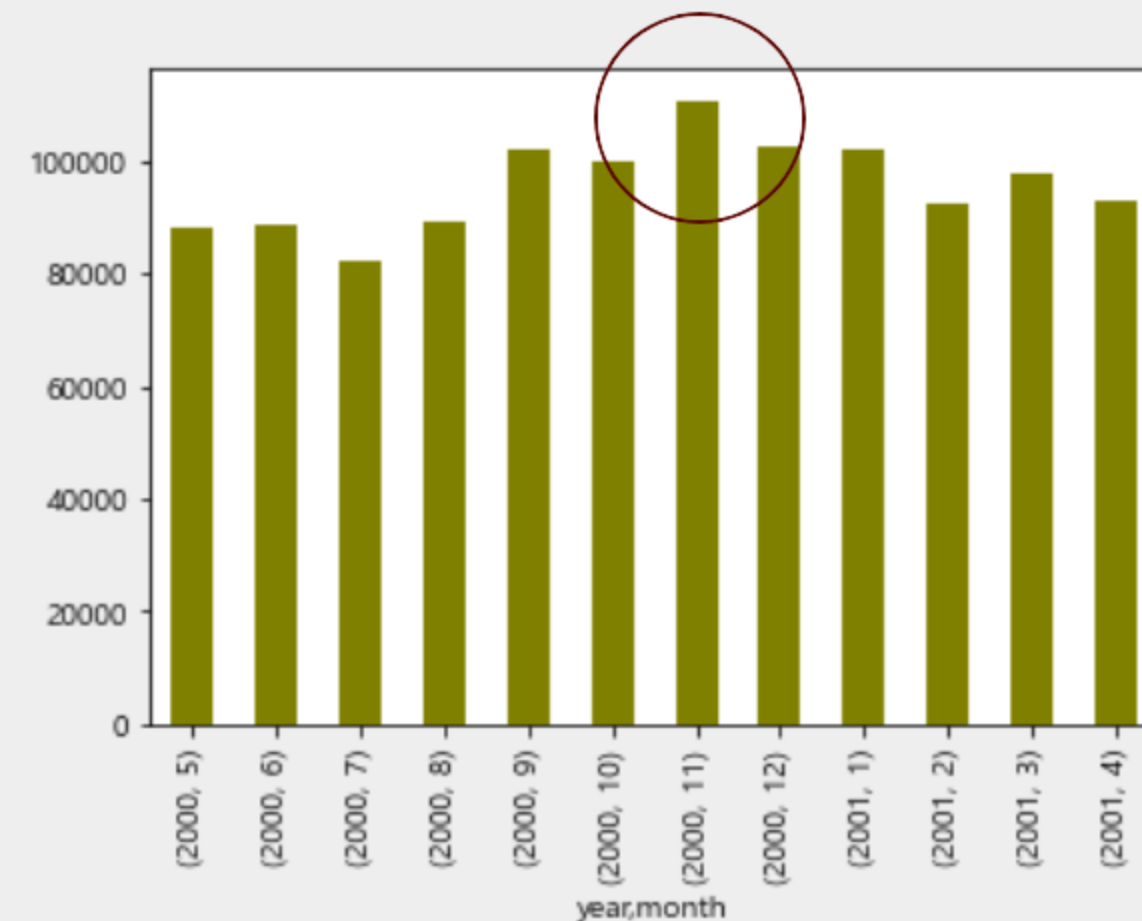
## 구매정보 데이터

월별 실구매액 탐색, 데이터 2000년 5월 ~ 2001년 4월 말까지 12개월의 데이터임.



월별 실구매액 합계

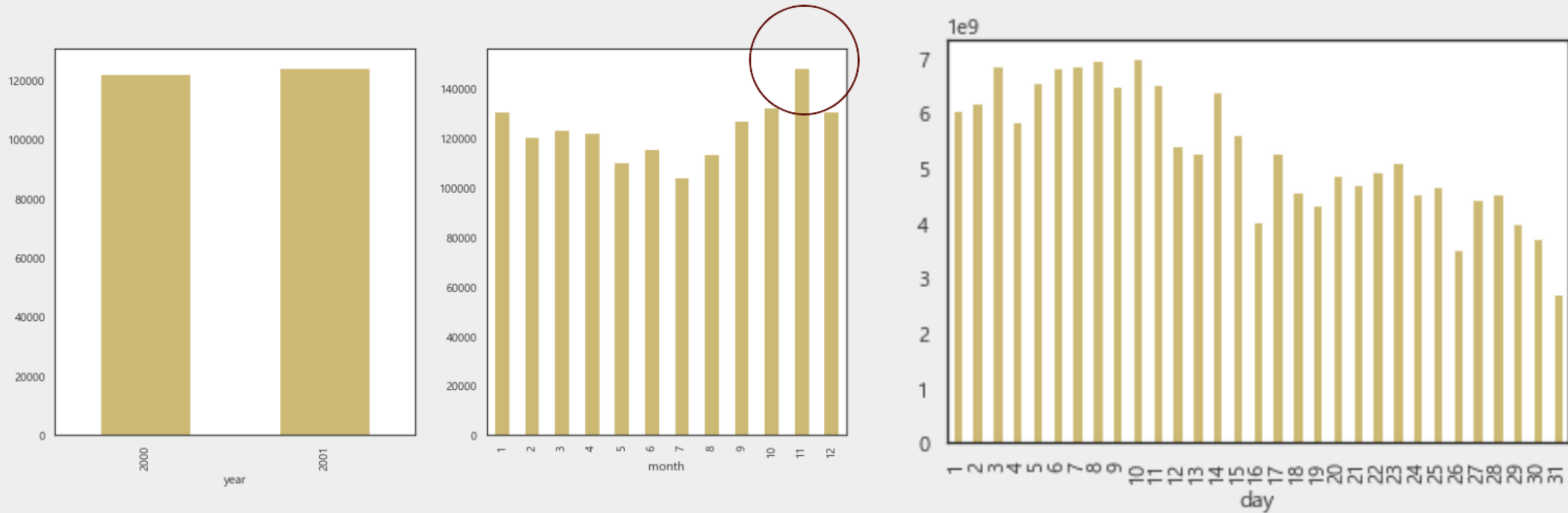
2000년 10월과 2001년 4월 실구매액 합계가 높음.



월별 실구매액 평균

2000년 11월 실구매액 평균이 높은것을 알 수 있음.

## 구매정보 데이터



### 년도별, 월별, 일별 평균구매금액

월 초가 월말보다 실구매액 평균 실구매액이 높고, 2000년도 보다 2001년 평균 실구매금액이 높아진 것을 알 수 있음. 12개월 중 11월이 평균금액 가장 높음.



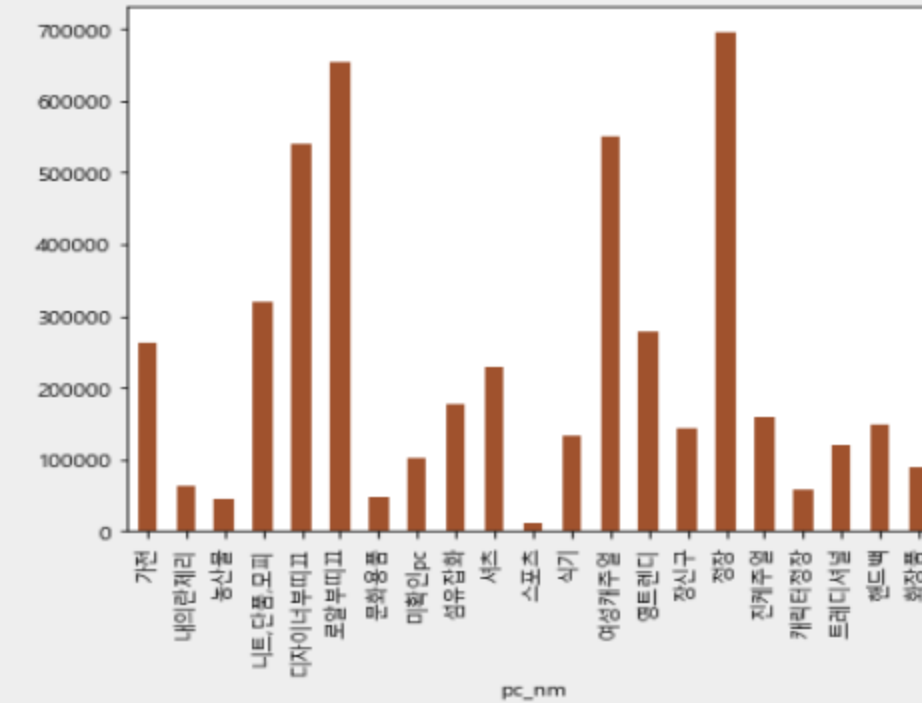
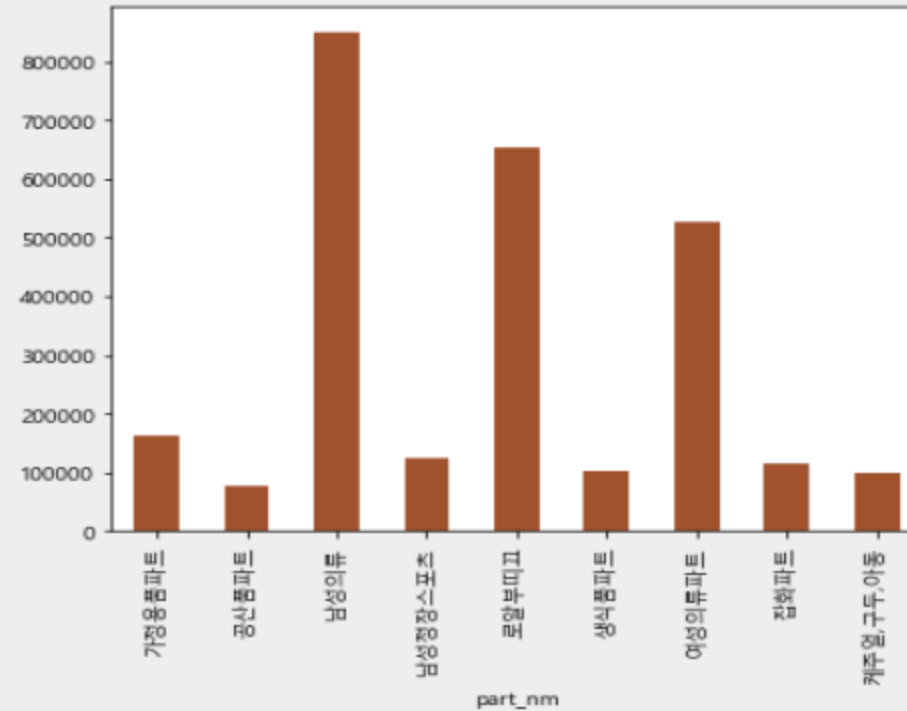
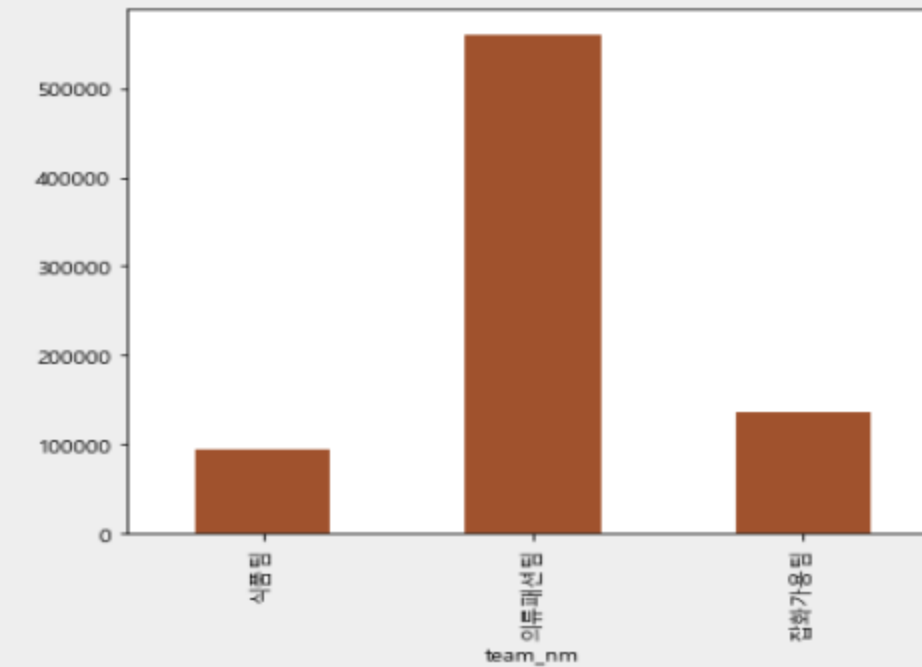
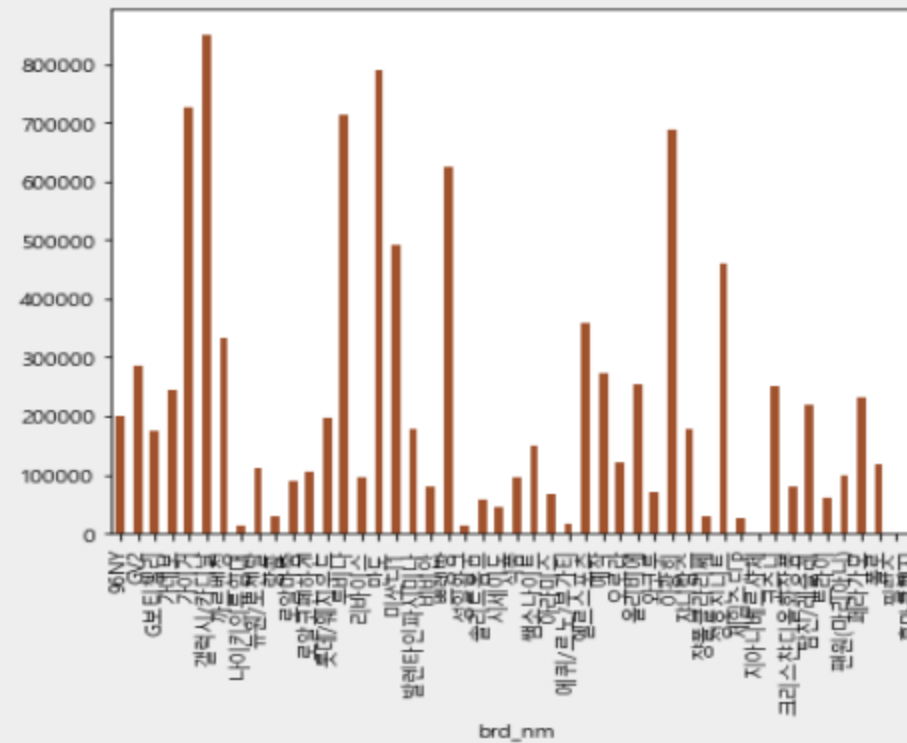
## 구매정보 데이터

구매액이 가장높은 고객의 브랜드코드, 팀코드, 파트코드, 상품군코드별 실구매금액 평균

> 의류패션팀의 실구매금액 평균이 가장 높음.

> 파트코드 또한 남성의류, 로얄부띠끄, 여성의류파트가 높은것으로 보아 패션쪽 구매를 많이 하는 여성 고객인 것으로 보임.

> 상품군 코드 또한 정장, 여성캐주얼 등 의류가 높음.

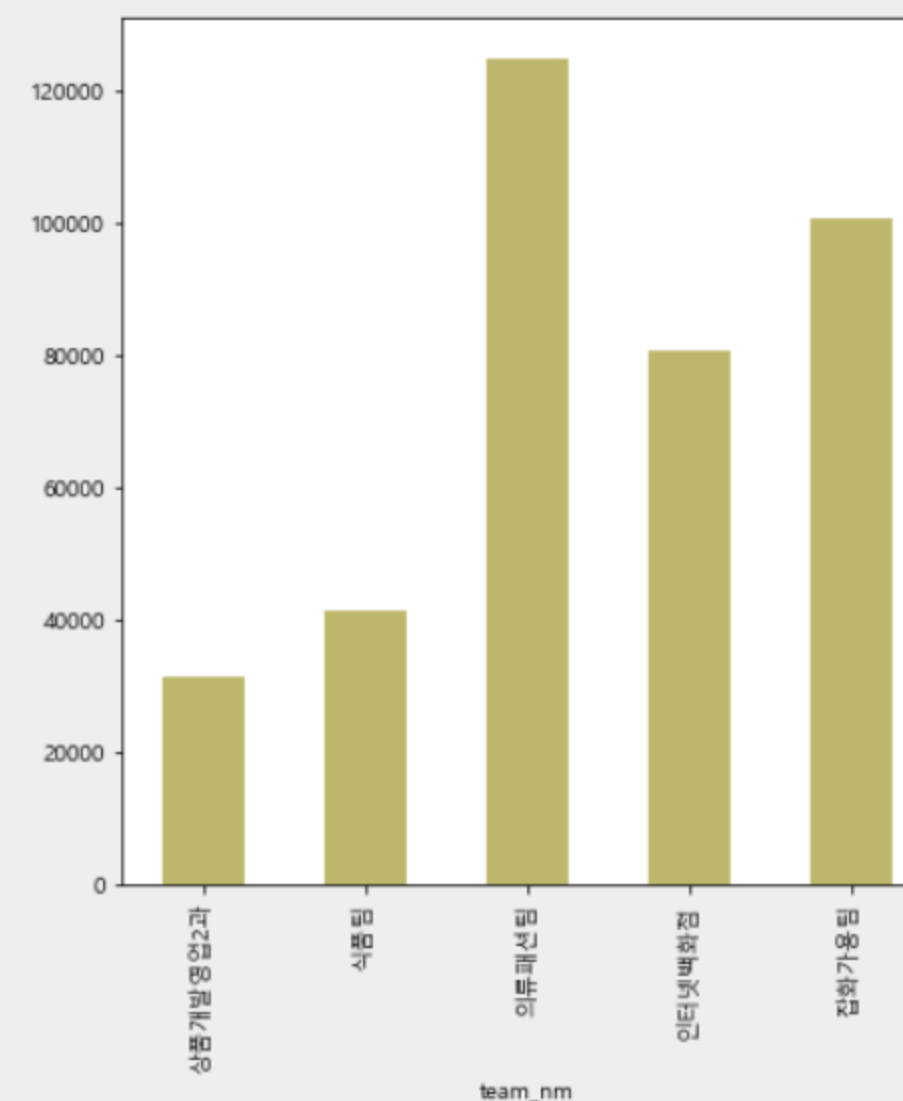
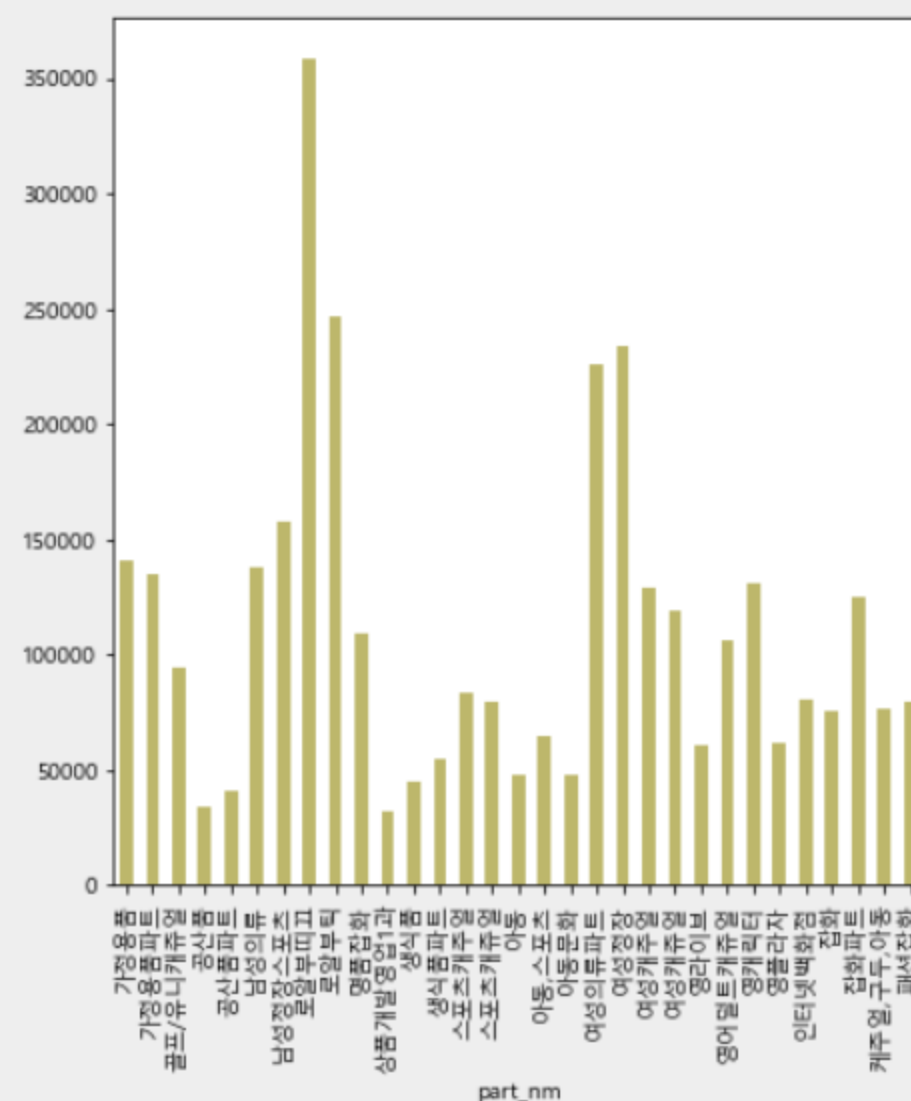


## 구매정보 데이터

전체 고객의 파트별 평균 실구매액

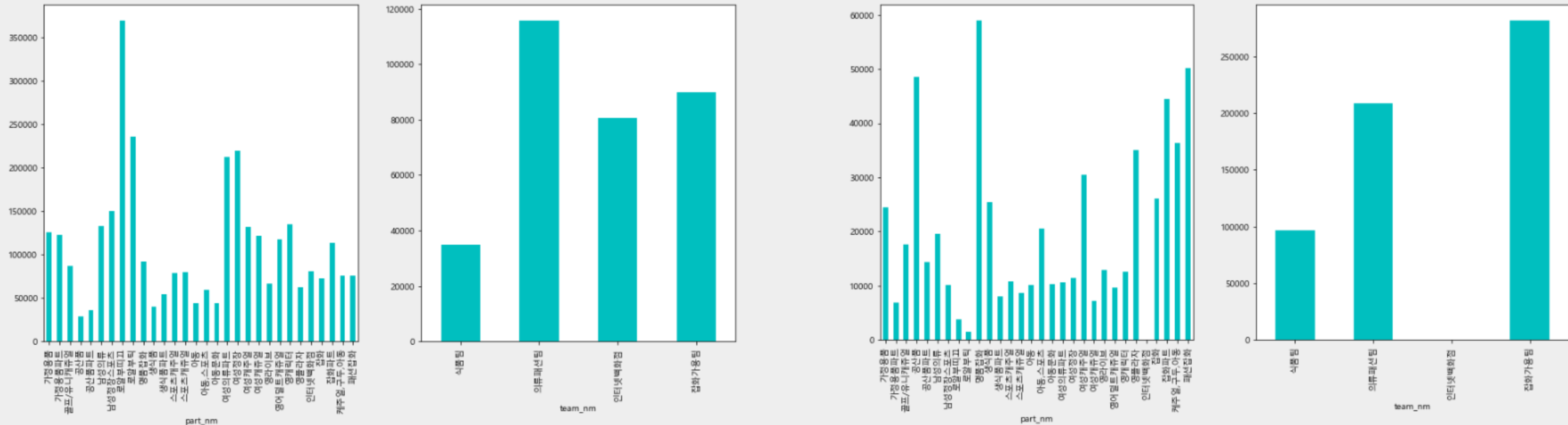
> 전체 고객의 1년 평균 실구매액이 가장 높은 팀은 의류패션팀이고 2위는 잡화가용팀.

> 로얄 부띠끄에서 평균 실구매액이 높은데, 가격대가 높기 때문인 것으로 유추됨.



# 구매정보 데이터

미혼 고객의 데이터 살펴보기.



## 파트별, 팀별 구매고객평균 과 구매횟수

미혼고객의 평균실구매액이 가장 높은 파트는 로얄부띠끄 인 것을 알 수 있고, 팀파트에서도 의류패션팀의 평균 실구매액이 가장 높고, 잡화가용팀, 인터넷백화점, 식품팀 순인 것을 확인할 수 있음.

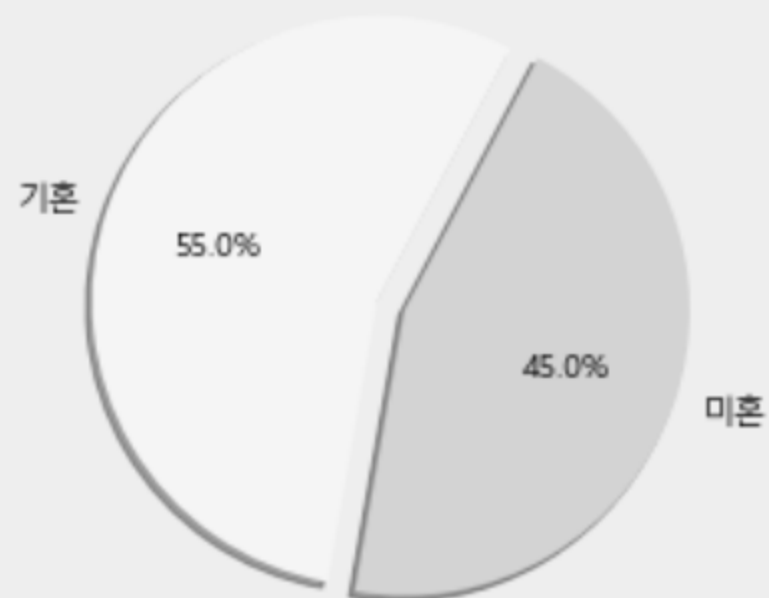
구매횟수는 명품잡화, 공산품순으로 높음.

잡화가용팀, 의류패션팀 순으로 구매횟수가 많은 것을 확인할 수 있음.

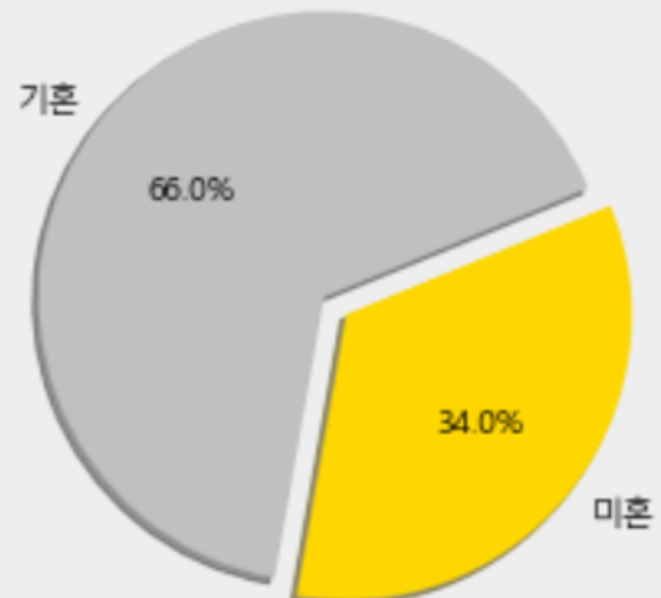


## 구매정보 데이터

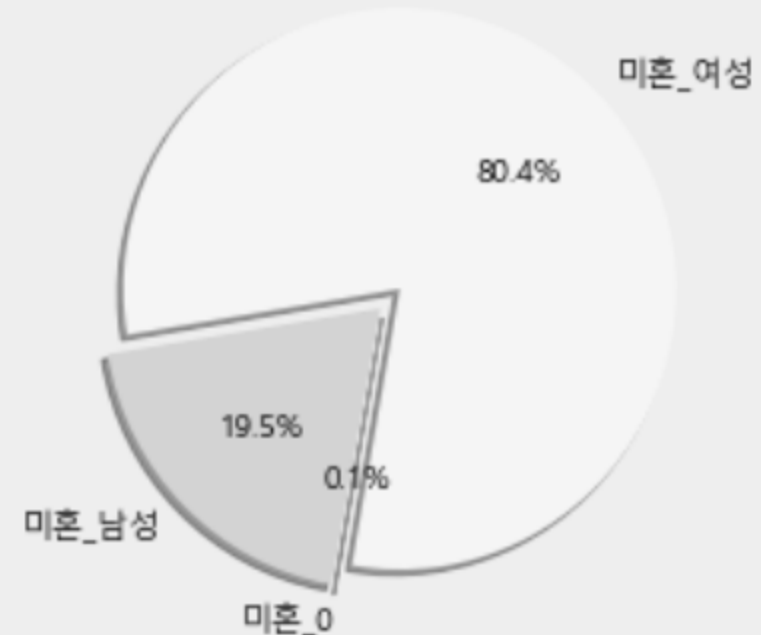
결혼여부별 구매고객 수 비율



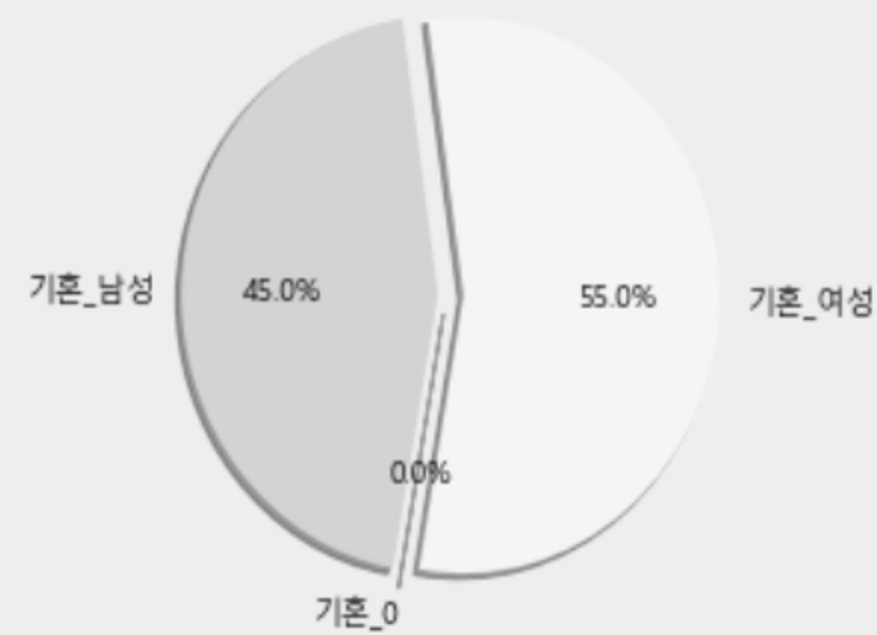
구매고객 결혼여부별 구매횟수 비율



미혼고객 남녀 비율

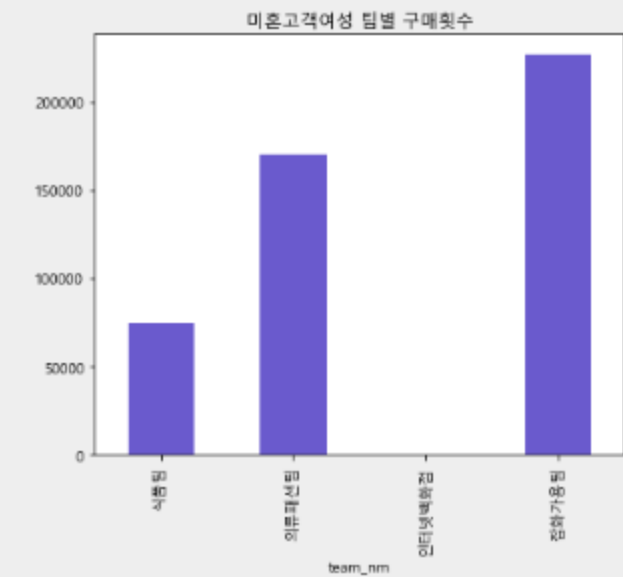
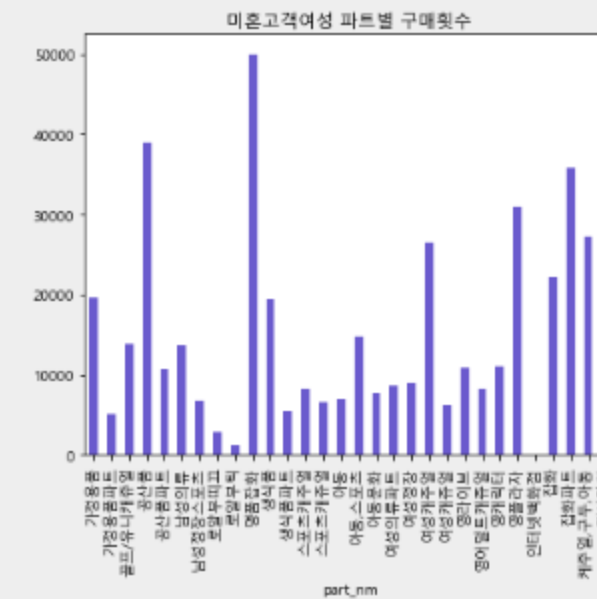
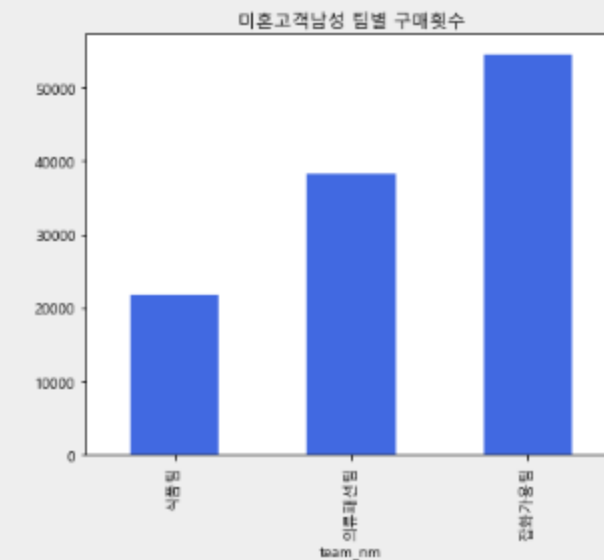
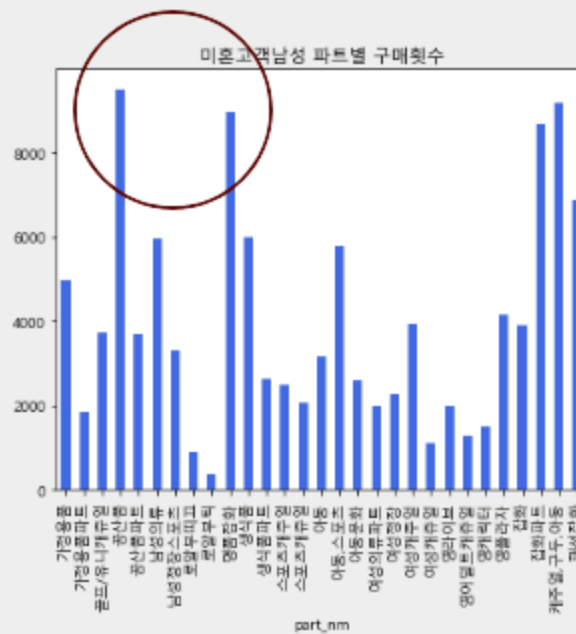
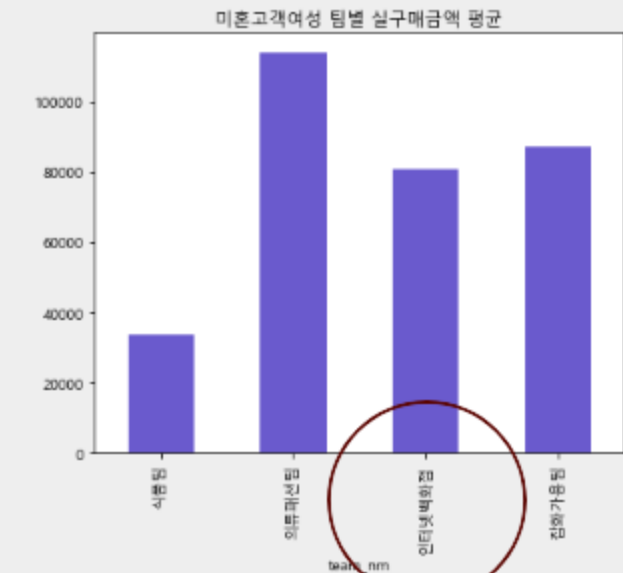
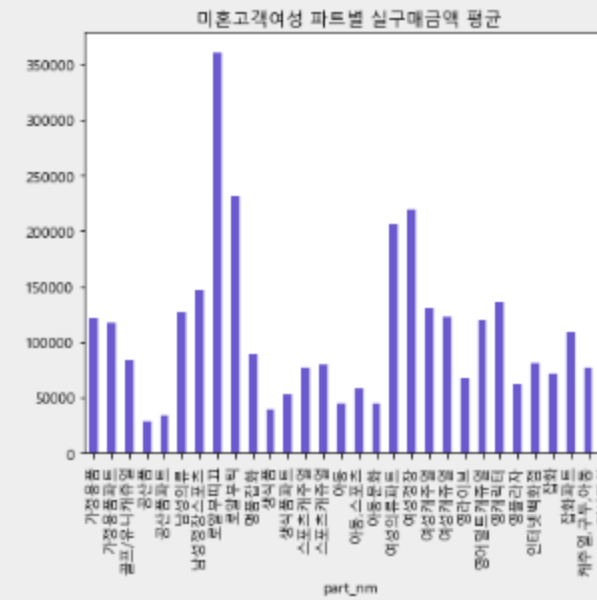
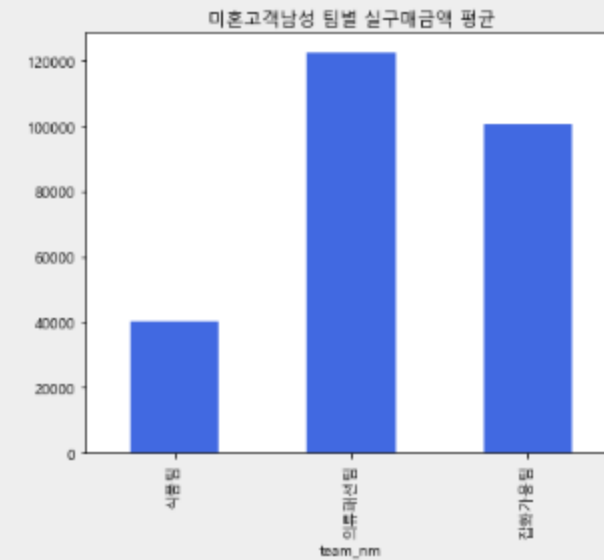
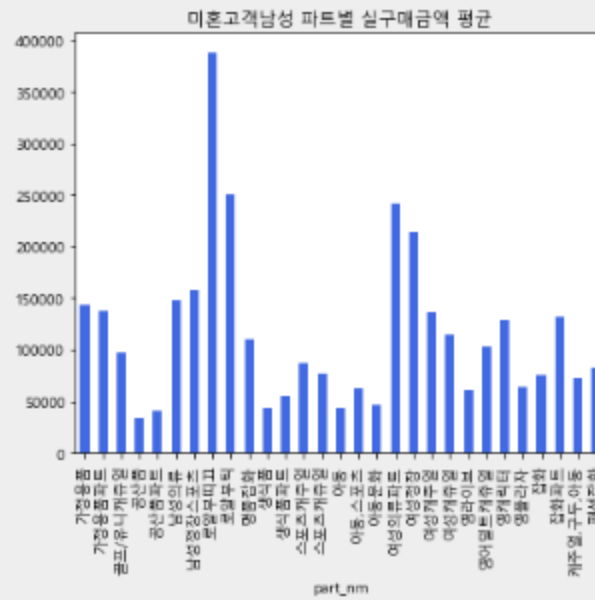


기혼고객 남녀 비율



# 구매정보 데이터

미혼 고객의 데이터 살펴보기.

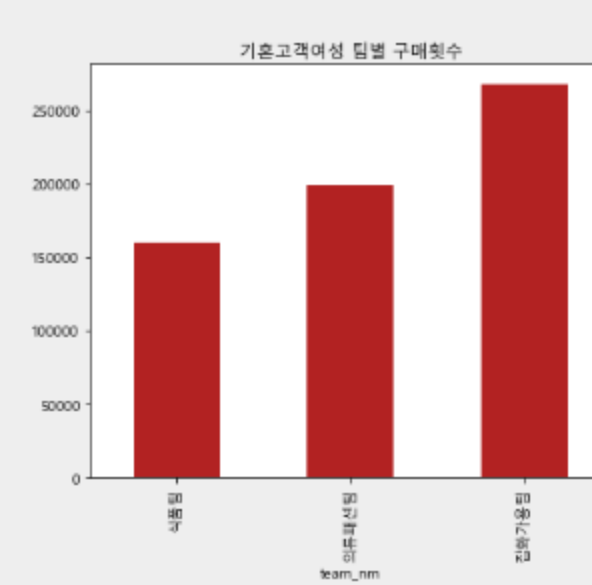
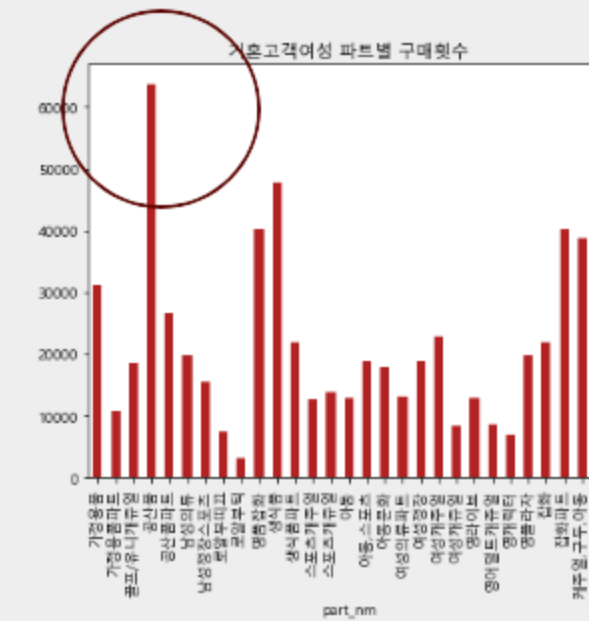
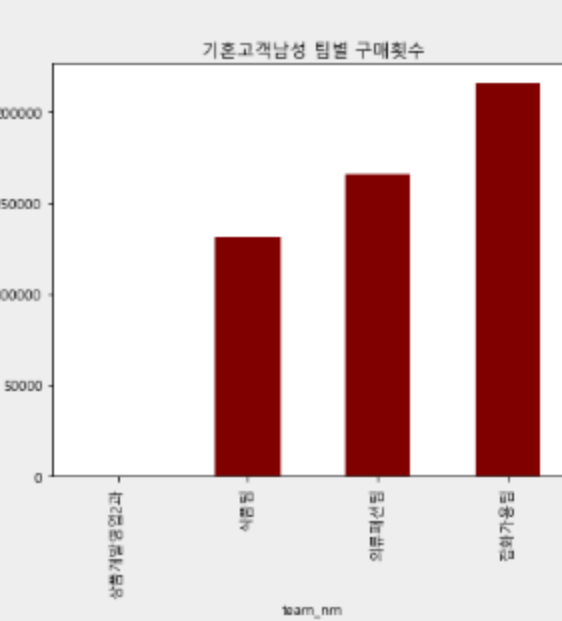
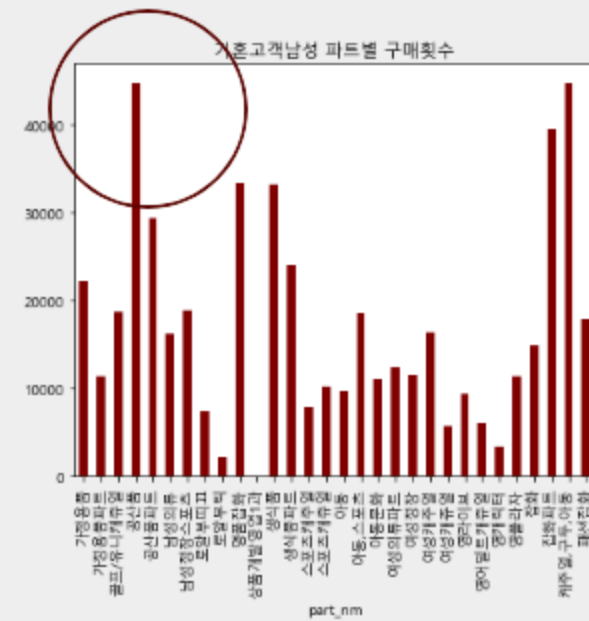
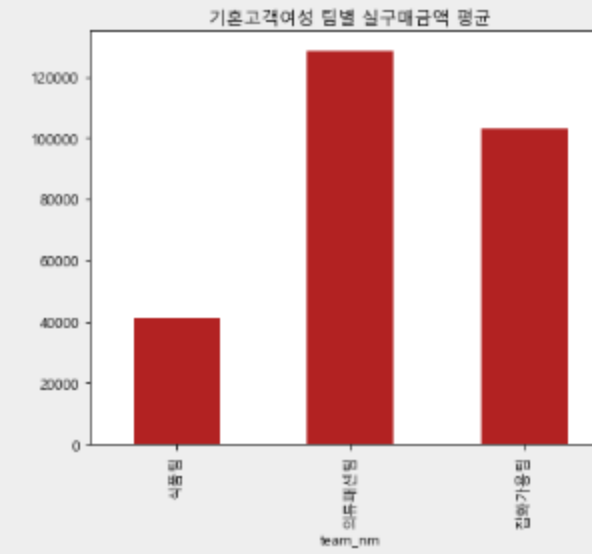
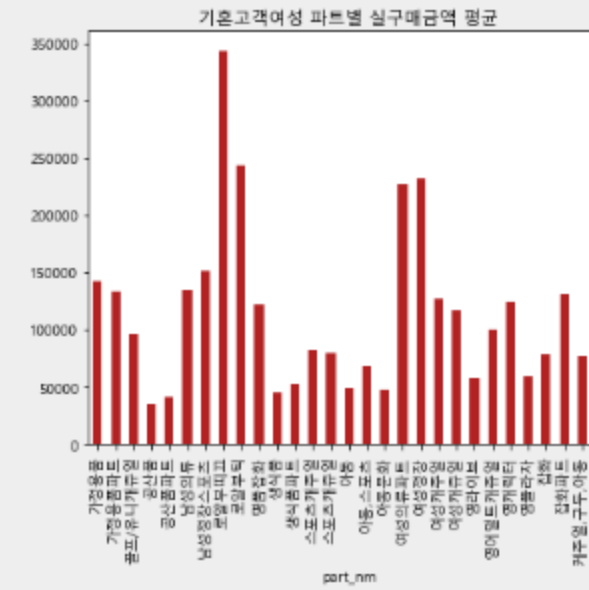
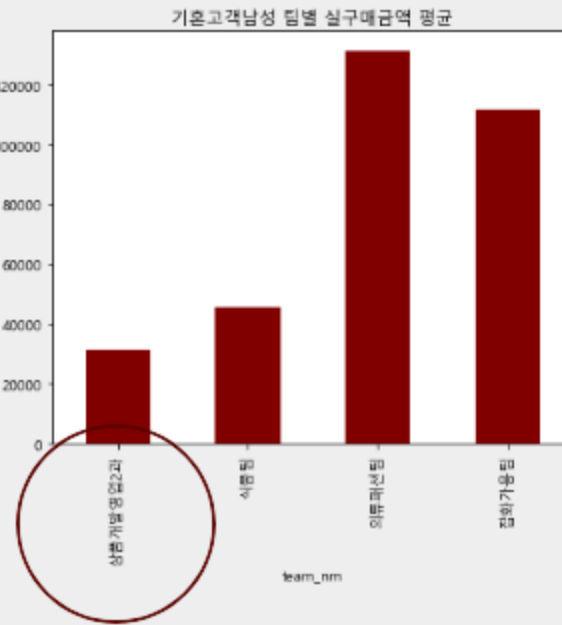
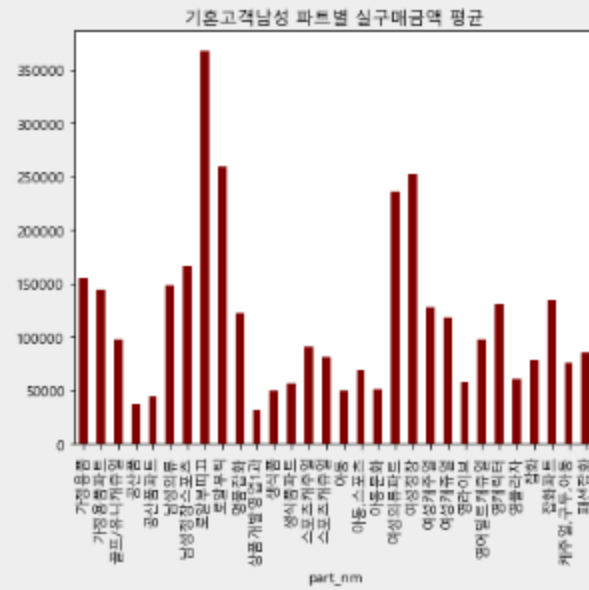


미혼전체 : (548372, 25) / 미혼남성 : (106988, 25) / 미혼여성 : (441012, 25)

미혼 여성의 구매정보가 남성보다 많음.

# 구매정보 데이터

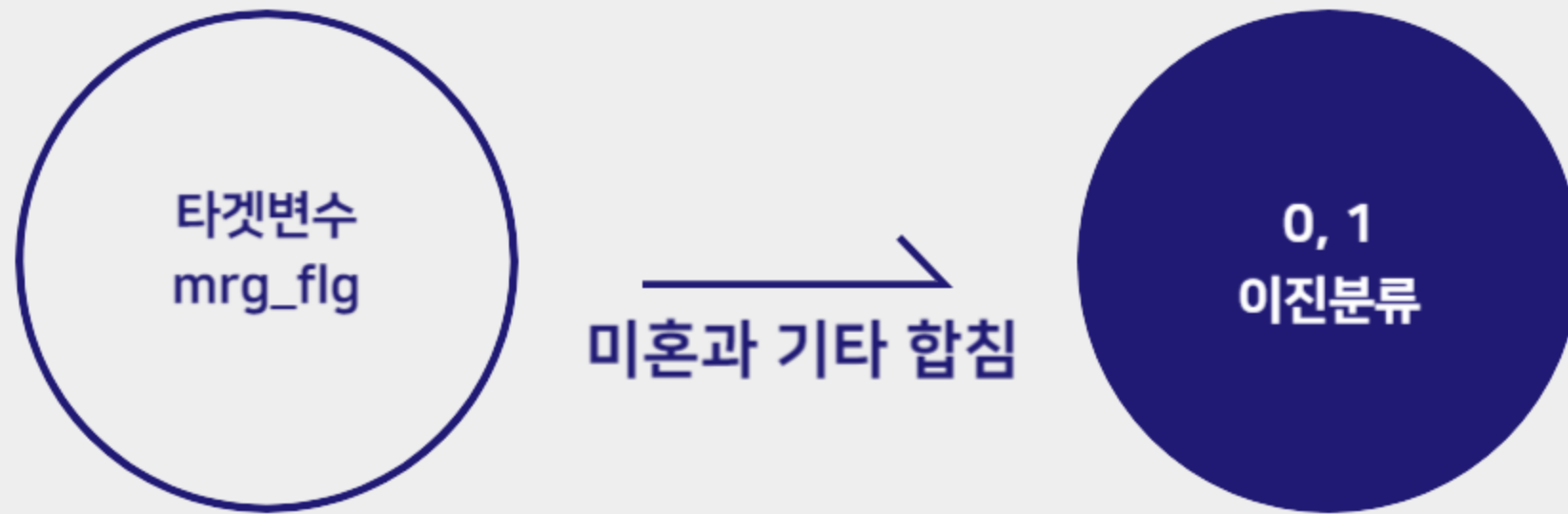
기혼 고객의 데이터 살펴보기.



기혼전체 : (1062117, 25) / 기혼남성 : (477984, 25) / 기혼여성 : (583704, 25)  
남성과 여성의 구매정보 관측치수는 남녀비율과 비슷함.

## 타겟 변수 확인

결혼여부 예측을 위해 결혼여부코드를 타겟변수로 설정.



```

1 # 타겟변수 mrg_flg
2
3 customers.mrg_flg.value_counts()
4 # 기혼 : 24,479, 미혼 + 기타 : 22,516

```

```

27479
17801
4714
1
Name: mrg_flg, dtype: int64

```

```

1 def target_binary(x):
2     if x == 1: return x
3     else: return 0
4
5 customers['mrg_flg_gb'] = customers.mrg_flg.map(target_binary)

```

```

1 customers.mrg_flg_gb.value_counts()

```

```

27479
22516
Name: mrg_flg_gb, dtype: int64

```

# 파생 변수 생성

구매정보 데이터를 고객정보데이터와 합침 (고객아이디 기준으로)

```
1 data_s = df_m.groupby(["custid"])[['tot_amt', 'dis_amt', 'net_amt']].agg(['mean', 'sum', 'count'])
2 data_s
```

C:\Users\User\AppData\Local\Temp\ipykernel\_7616\2819514195.py:1: FutureWarning: Indexing with multiple levels is deprecated, use a list instead.

```
data_s = df_m.groupby(["custid"])[['tot_amt', 'dis_amt', 'net_amt']].agg(['mean', 'sum', 'count'])
```

|        | tot_amt  |         |       | dis_amt |       |       | net_amt  |         |       |
|--------|----------|---------|-------|---------|-------|-------|----------|---------|-------|
|        | mean     | sum     | count | mean    | sum   | count | mean     | sum     | count |
| custid |          |         |       |         |       |       |          |         |       |
| 1      | 49356.90 | 3800481 | 77    | 546.75  | 42100 | 77    | 48810.14 | 3758381 | 77    |
| 2      | 76154.61 | 2132329 | 28    | 2541.07 | 71150 | 28    | 73613.54 | 2061179 | 28    |
| 3      | 95465.28 | 6491639 | 68    | 964.56  | 65590 | 68    | 94500.72 | 6426049 | 68    |
| 4      | 68233.33 | 409400  | 6     | 4683.33 | 28100 | 6     | 63550.00 | 381300  | 6     |

```
1 # 구매일자 cnt
2 data1 = df_m.groupby(["custid", 'sales_date'])['tot_amt'].count()
3 data1 = pd.DataFrame(data1)
4 date_c = pd.DataFrame(data1.reset_index().drop('tot_amt', axis=1).groupby(["custid"])[['custid']].count())
5 date_c.columns = ['date_c']
6 date_c.reset_index()
```

| custid | date_c |
|--------|--------|
| 0      | 1      |
| 1      | 2      |
| 2      | 3      |
| 3      | 4      |

```
1 # 구매상품 중복제거 개수 cnt
2 data1 = df_m.groupby(["custid", 'goodcd'])['tot_amt'].count()
3 data1 = pd.DataFrame(data1)
4 item_c = pd.DataFrame(data1.reset_index().drop('tot_amt', axis=1).groupby(["custid"])[['custid']].count())
5 item_c.columns = ['item_c']
6 item_c.reset_index()
```

| custid | item_c |
|--------|--------|
| 0      | 1      |
| 1      | 2      |
| 2      | 3      |
| 3      | 4      |
| 4      | 5      |
| ...    | ...    |
| 19990  | 49996  |
| 19991  | 49997  |
| 19992  | 49998  |
| 19993  | 49999  |

## 1. 구매액

구매액, 할인액, 실구매액의  
최대, 최소, 평균 변수를  
고객 아이디 기준으로 생성.

## 2. 판매일자

판매일자를 이용하여  
12개월 동안 고객들의  
구매일수를 변수로 생성.

## 3. 구매상품 종류 수

구매상품 중복제거하여  
12개월동안 얼마나 다양한  
상품을 구매하였는지 변수로 생성.



# 파생 변수 생성

구매정보 데이터를 고객정보데이터와 합침 (고객아이디 기준으로)

```
1 # 월말(2001-04-30) 과 마지막구매일자 차이 일수
2
3 df_m['sales_date'].max()

Timestamp('2001-04-29 00:00:00')
```

```
1 data1 = df_m.groupby(["custid"])[ 'sales_date' ].max()
2 data1 = pd.DataFrame(data1)
3 data1['std_date'] = '2001-04-30'
4 data1['std_date'] = pd.to_datetime(data1['std_date'])
5 data1['date_diff'] = data1['std_date'] - data1['sales_date']
6 date_c['date_diff'] = data1['date_diff'].dt.days
```

```
1 date_c.head()
```

|        | date_c | date_diff | item_c |
|--------|--------|-----------|--------|
| custid |        |           |        |
| 1      | 41     | 1         | 29     |
| 2      | 11     | 19        | 17     |
| 3      | 27     | 3         | 37     |

## 4. 마지막 구매일까지의 일자

마지막월(2001년 4월) 말일을 기준으로 마지막 구매일자와의 차이 일수를 구하여 변수생성.

```
1 # 월별 구매여부
2 data1 = df_m.groupby(["custid", "month"])[ 'sales_date' ].count()
3 data1 = pd.DataFrame(data1)
4 df_pivot = data1.reset_index().drop('sales_date', axis=1)
5 df_pivot['cnt'] = '1'
6 df_pivot = df_pivot.pivot(index='custid', columns='month', values='cnt').fillna('0')
```

```
1 df_pivot.head()
```

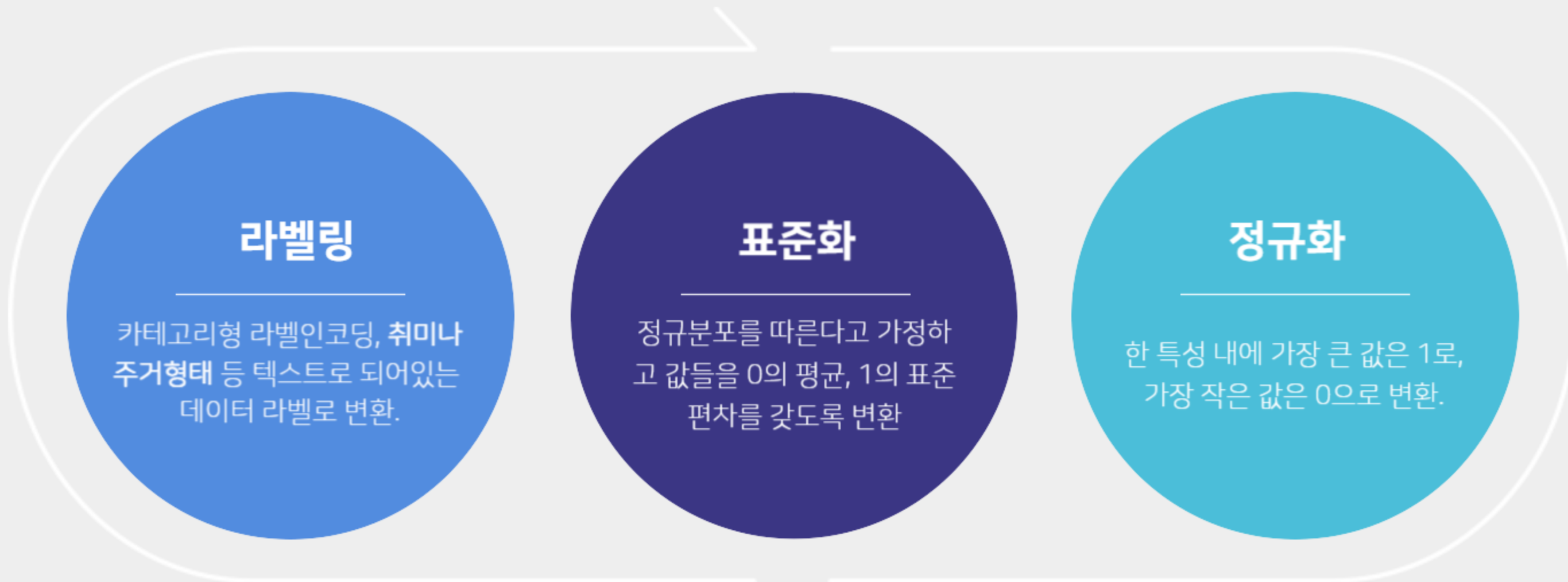
|       | month_1 | month_2 | month_3 | month_4 | month_5 | month_6 | month_7 | month_8 | month_9 | month_10 | month_11 | month_12 |
|-------|---------|---------|---------|---------|---------|---------|---------|---------|---------|----------|----------|----------|
| ustid |         |         |         |         |         |         |         |         |         |          |          |          |
| 1     | 1       | 1       | 1       | 1       | 1       | 1       | 1       | 1       | 1       | 1        | 1        | 1        |
| 2     | 0       | 1       | 1       | 1       | 0       | 0       | 1       | 1       | 0       | 0        | 0        | 0        |
| 3     | 1       | 0       | 1       | 1       | 0       | 1       | 1       | 1       | 0       | 1        | 0        | 1        |
| 4     | 1       | 0       | 0       | 1       | 0       | 0       | 0       | 0       | 0       | 0        | 0        | 1        |
| 5     | 0       | 0       | 0       | 1       | 0       | 0       | 0       | 0       | 0       | 0        | 0        | 0        |

## 5. 월별 구매 현황

12개월 동안 해당월 구매기록이 있으면 '1' 없으면 '0'으로 더미 데이터 생성.

## 변수 표준화 및 정규화

타겟 변수를 더 정확하게 예측하기 위해 변수 스케일 맞춰주고 모델에 입력될 수 있도록 전처리 하는 과정.



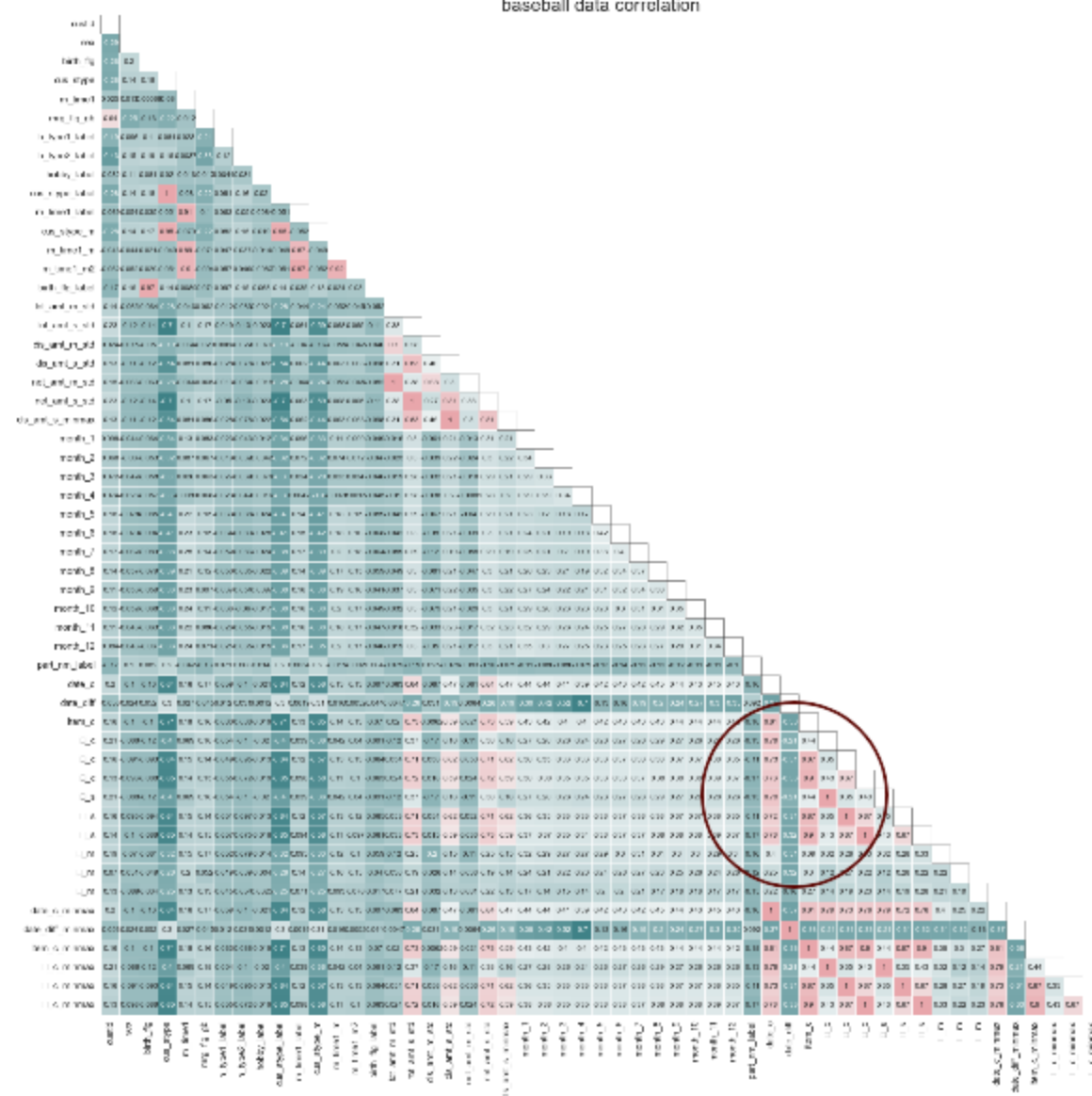
특성들의 단위를 무시할 수 있도록, 특성들의 값의 범위를 비슷하게 만들어 줌.



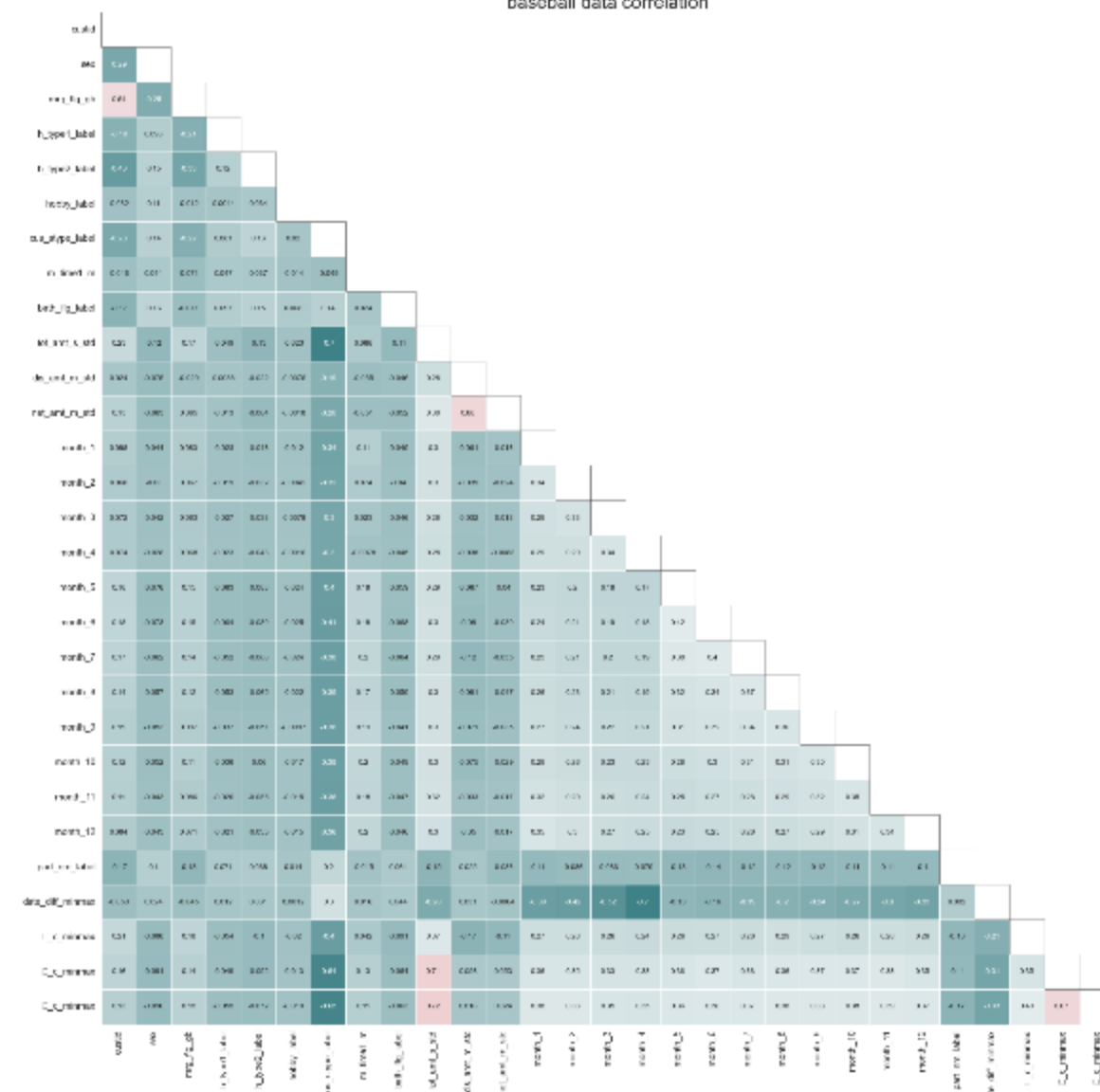
# 변수 선정

상관관계, 다중공선성 확인

baseball data correlation



baseball data correlation





# 변수 선정

상관관계, 다중공선성 확인

```
1 # 다중공선성 확인
2 from statsmodels.stats.outliers_influence import variance_inflation_factor
3
4 # VIF 출력을 위한 데이터 프레임 형성
5 vif = pd.DataFrame()
6
7 # VIF 값과 각 Feature 이름에 대해 설정
8 vif["VIF Factor"] = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
9 vif["features"] = X.columns
10
11 # VIF 값이 높은 순으로 정렬
12 vif = vif.sort_values(by="VIF Factor", ascending=False)
13 vif = vif.reset_index().drop(columns='index')
14 vif
```

|     | VIF Factor | features        |
|-----|------------|-----------------|
| 0   | 20.39      | m_time1_m       |
| 1   | 15.21      | cus_stype_label |
| 2   | 14.22      | sex             |
| 3   | 5.90       | hobby_label     |
| 4   | 5.45       | 잡_c_minmax      |
| 5   | 5.06       | month_4         |
| 6   | 4.74       | custid          |
| 7   | 4.73       | h_type2_label   |
| 8   | 4.54       | 의_c_minmax      |
| 9   | 4.27       | birth_flg_label |
| ... | .....      | .....           |

## 다중공선성 (Multicollinearity)

> 일반적으로 회귀모형에서 설명변수간에 정확한 선형관계(완전 공선성)는 나타나지 않으며, 단지 그 상관관계가 높게 나타나는 문제가 발생하는데, 이경우를 다중공선성(multicollinearity)라 함.

VIF란, Variance Inflation Factor의 약자.

> VIF는 독립변수가 여러개 있을 때, 특정 독립변수를 종속변수로하고 나머지 독립변수를 독립변수로 하여 회귀분석을 수행하여 변수간에 관계성을 측정

일반적으로 VIF값이 10이상이면 문제가 있을 수 있다고 판단하고, 제외  
=> vif 값이 15 이상인 m\_time1\_m (주구매 시간대 minmax 삭제)

## 분류모델링

이진분류, 분류모델 + 로지스틱 회귀



## 모델 평가

- 모델 평가를 위한 metrics

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
```

```
1 # 평가 함수
2 def scores(y_test, pred):
3     # f1스코어
4     print("f1스코어 : ", f1_score(y_test, pred))
5     # 정확도
6     print("정확도 : ", accuracy_score(y_test, pred))
7     # 정밀도
8     print("정밀도 : ", precision_score(y_test, pred))
9     # 재현율
10    print("재현율 : ", recall_score(y_test, pred))
```

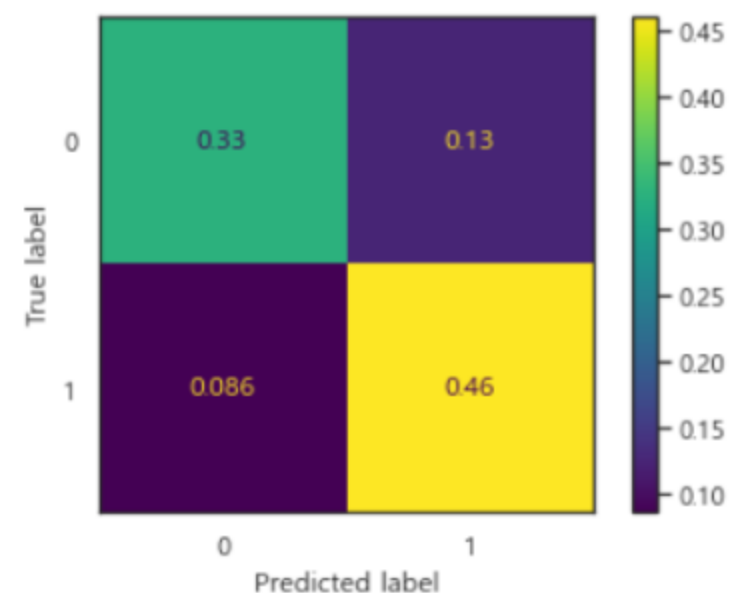
0.9s

### 평가함수 이용

```
1 plot_confusion_matrix(best_dclf, X_test, y_test, normalize='all')
2 # 1은 기혼
```

Warning: The function `plot_confusion_matrix` is deprecated in 1.0 and will be removed in 1.2. Use `ConfusionMatrixDisplay.from_predictions` or `ConfusionMatrixDisplay.from_predictions` instead.

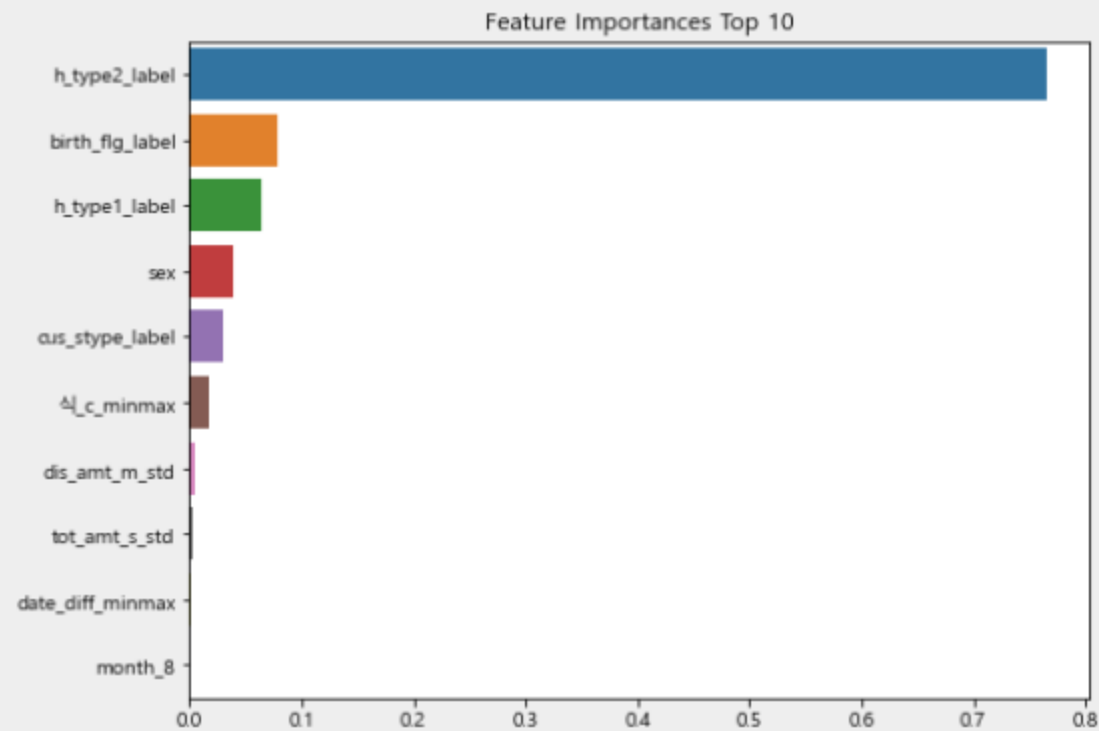
sklearn.metrics.\_plot.confusion\_matrix.ConfusionMatrixDisplay at 0x21...



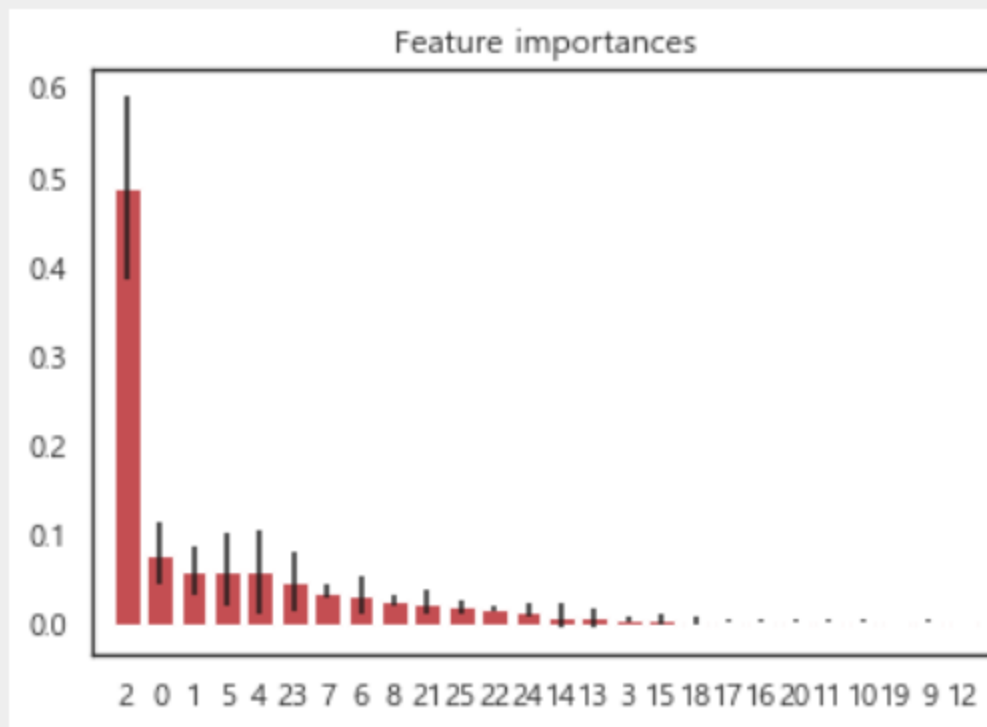
### ConfusionMatrixDisplay

# 모델 평가

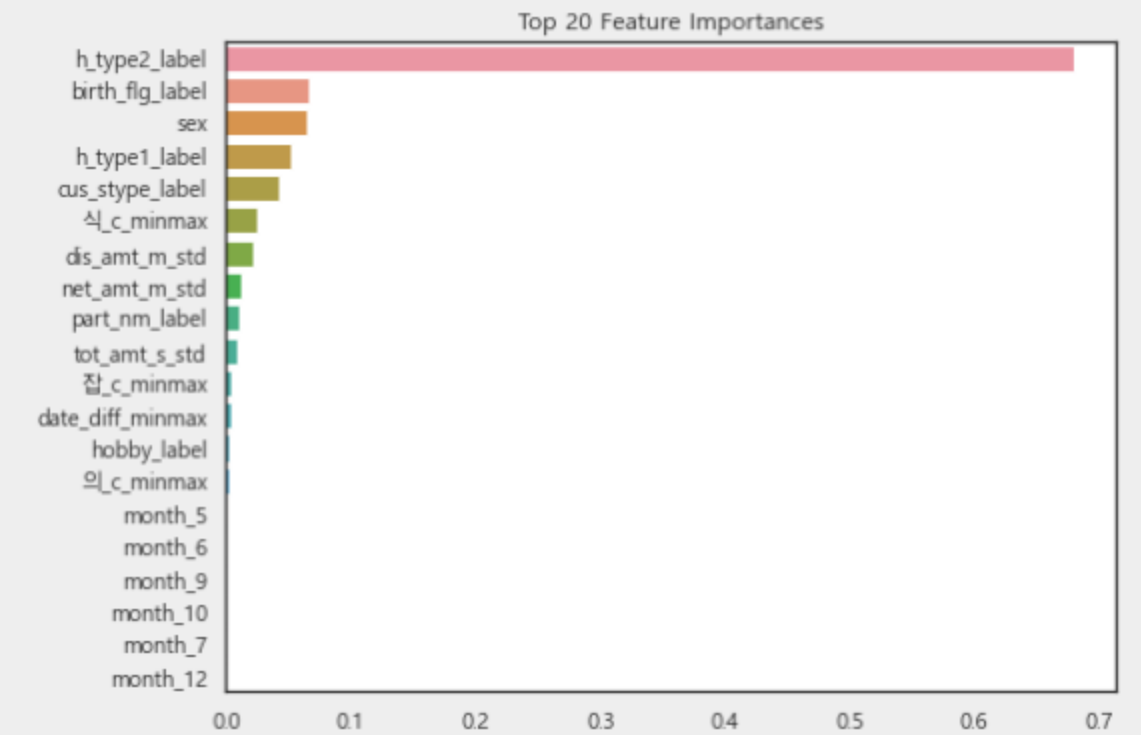
중요변수 확인, `best_gclf.feature_importances_`



DecisionTreeClassifier  
변수중요도



RandomForest  
변수중요도



GradientBoostingClassifier  
변수중요도

# 모델 평가 예측 결과 평가

1) DecisionTreeClassifier

```
1 from sklearn.model_selection import GridSearchCV
2
3 params = {
4     'max_depth': [2, 3, 5, 10],
5     'min_samples_split': [2, 3, 5],
6     'min_samples_leaf': [1, 5, 8]
7 }
8
9 clf = DTC(splitter='best')
10
11 grid_clf = GridSearchCV(clf, param_grid=params, scoring='accuracy', cv=5)
12 grid_clf.fit(X_train, y_train)
13
14 print('GridSearchCV 최적 하이퍼 파라미터:', grid_clf.best_params_)
15 print('GridSearchCV 최고 정확도:', grid_clf.best_score_)
✓ 27.7s
```

GridSearchCV 최적 하이퍼 파라미터: {'max\_depth': 5, 'min\_samples\_leaf': 5, 'min\_samples\_split': 2}  
GridSearchCV 최고 정확도: 0.789303700895702

f1스코어 : 0.8169784680550654

정확도 : 0.7925792579257925

정밀도 : 0.7946781115879828

재현율 : 0.8405665516615217

4) 로지스틱회귀

```
1 # 파라메터 후보
2 param_grid = [{"C": np.logspace(-3, 3, 7), "penalty": ["l1", "l2"]}
3
4 rlc1f = LR(max_iter=1000)
5 # 그리드 서치 진행
6 grid_search = GridSearchCV(rlc1f, param_grid, cv=5)
7 grid_search.fit(X_train, y_train)
8
9 print('최적 하이퍼 파라미터:\n', grid_search.best_params_)
10 print('최고 예측 정확도: {:.4f}'.format(grid_search.best_score_))
```

f1스코어 : 0.7443289652056032

정확도 : 0.7170717071707171

정밀도 : 0.7369362920544023

재현율 : 0.7518714624794596

2) RandomForest

```
1 # 무작위 학습
2 rdlf = RFC(n_estimators=5, random_state=0)
3 rdlf.fit(X_train, y_train)
```

```
RandomForestClassifier
RandomForestClassifier(n_estimators=5, random_state=0)
```

```
1 params = [ 'n_estimators': [10, 100],
2           'max_depth': [6, 8, 10, 12],
3           'min_samples_leaf': [8, 12, 18],
4           'min_samples_split': [8, 16, 20]
5         ]
6
7 # RandomForestClassifier 객체 생성 후 GridSearchCV 수행
8 rf_clf = RFC(random_state=0, n_jobs=-1)
9 grid_cv = GridSearchCV(rf_clf, param_grid=params, cv=3, n_jobs=-1)
10 grid_cv.fit(X_train, y_train)
11
12 print('최적 하이퍼 파라미터: ', grid_cv.best_params_)
13 print('최고 예측 정확도: {:.4f}'.format(grid_cv.best_score_))
```

f1스코어 : 0.8136848602416014

정확도 : 0.7886788678867886

정밀도 : 0.7868349249658936

재현율 : 0.8424319883147708

3) GradientBoostingClassifier

```
1 gdlf = GBC()
2 gdlf.fit(X_train, y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier()
```

```
1 params = {
2     'n_estimators': [100, 500],
3     'learning_rate': [0.05, 0.1]
4 }
5
6 grid_cv = GridSearchCV(gdlf, param_grid=params, cv=3, verbose=1)
7 grid_cv.fit(X_train, y_train)
8 print('최적 하이퍼 파라미터:\n', grid_cv.best_params_)
9 print('최고 예측 정확도: {:.4f}'.format(grid_cv.best_score_))
```

Fitting 3 folds for each of 4 candidates, totalling 12 fits  
최적 하이퍼 파라미터:  
{'learning\_rate': 0.05, 'n\_estimators': 500}  
최고 예측 정확도: 0.8038

f1스코어 : 0.8247640885439633

정확도 : 0.8012801280128012

정밀도 : 0.7976799727055612

재현율 : 0.8537520540441847

## 최종선정 (XGBClassifier)

```
1 import xgboost as xgb
2
3 # 넘파이 형태의 학습 데이터 세트와 테스트 데이터를 DMatrix로 변환하는 예제
4 dtrain = xgb.DMatrix(data=X_train, label=y_train)
5 dtest = xgb.DMatrix(data=X_test, label=y_test)
6
✓ 0.1s
```

```
1 xg_clf = xgb.XGBClassifier()
2 xg_clf.fit(X_train, y_train)
3
✓ 3.1s
```

```
XGBClassifier
XGBClassifier(base_score=0.5, booster='gbtree', callbacks=None,
               colsample_bylevel=1, colsample_bynode=1, colsample_bytree=1,
               early_stopping_rounds=None, enable_categorical=False,
               eval_metric=None, feature_types=None, gamma=0, gpu_id=-1,
               grow_policy='depthwise', importance_type=None,
               interaction_constraints='', learning_rate=0.300000012,
               max_bin=256, max_cat_threshold=64, max_cat_to_onehot=4,
               max_delta_step=0, max_depth=6, max_leaves=0, min_child_weight=1,
               missing=nan, monotone_constraints=(), n_estimators=100,
               num_parallel_tree=1, predictor='auto', random_state=0,
```

```
1 y_pre_probs = xg_clf.predict(X_test)
✓ 0.1s
```

f1스코어 : 0.8260600159334337

정확도 : 0.8034803480348035

정밀도 : 0.8058721934369603

재현율 : 0.8472852732885419



## 최종모델 선정

최종적으로 정확도가 모델 평가 결과가 가장 좋은 XGBoost 모델 선정.



## 04 느낀점